

Neural Network and Deep Learning

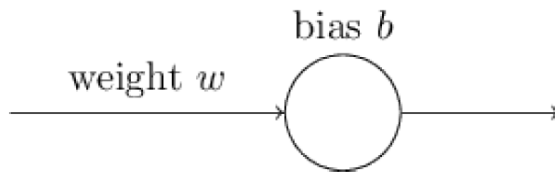
Big Data Analytics

Readings

- Neural Network and Deep Learning
 - Chap. 3

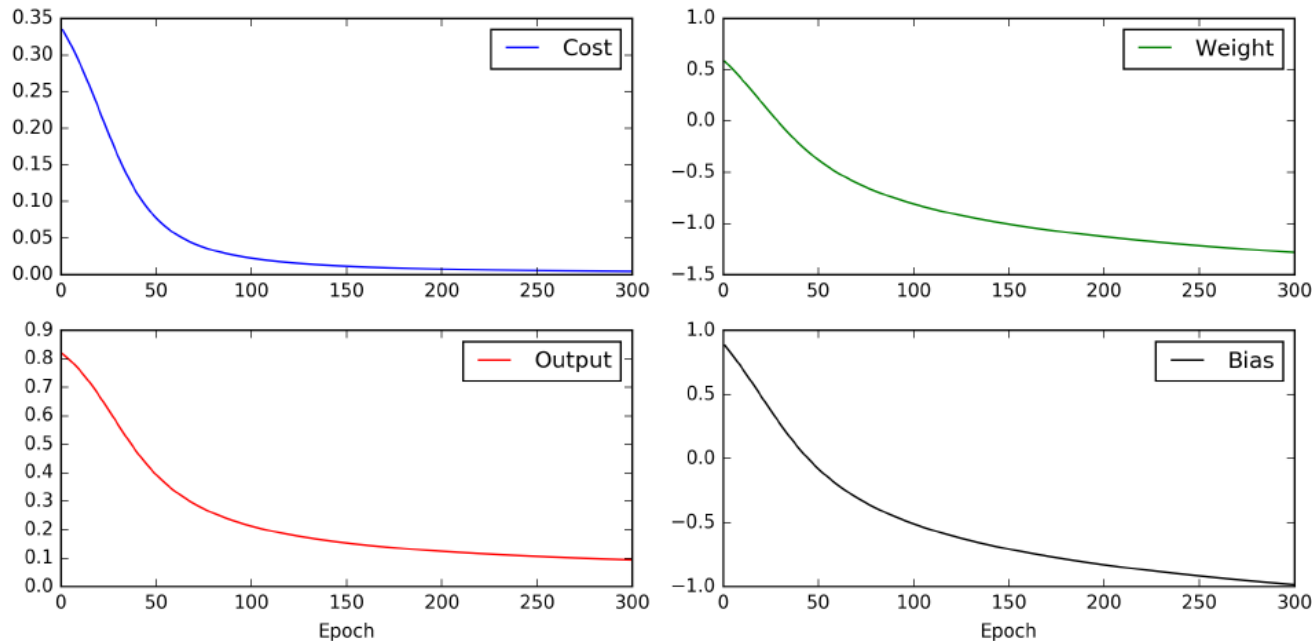
Cross-entropy cost function

- It'll be good if the neural network learns fast from it's errors
 - A toy network which take one input "1" and the desired output is "0"
 - Initial weight is "0.6" and initial bias is "0.9"
 - $z = "1.5"$ and $a = "0.82"$
 - $\eta=0.15$
 - Cost function is quadratic cost function



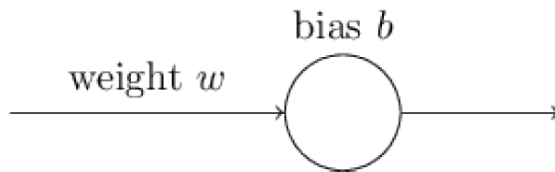
Cross-entropy cost function

- It'll be good if the neural network learns fast from it's errors
 - A toy network which take one input "1" and the desired output is "0"
 - Initial weight is "0.6" and initial bias is "0.9"
 - $z = "1.5"$ and $a = "0.82"$
 - $\eta = 0.15$
 - Cost function is quadratic cost function



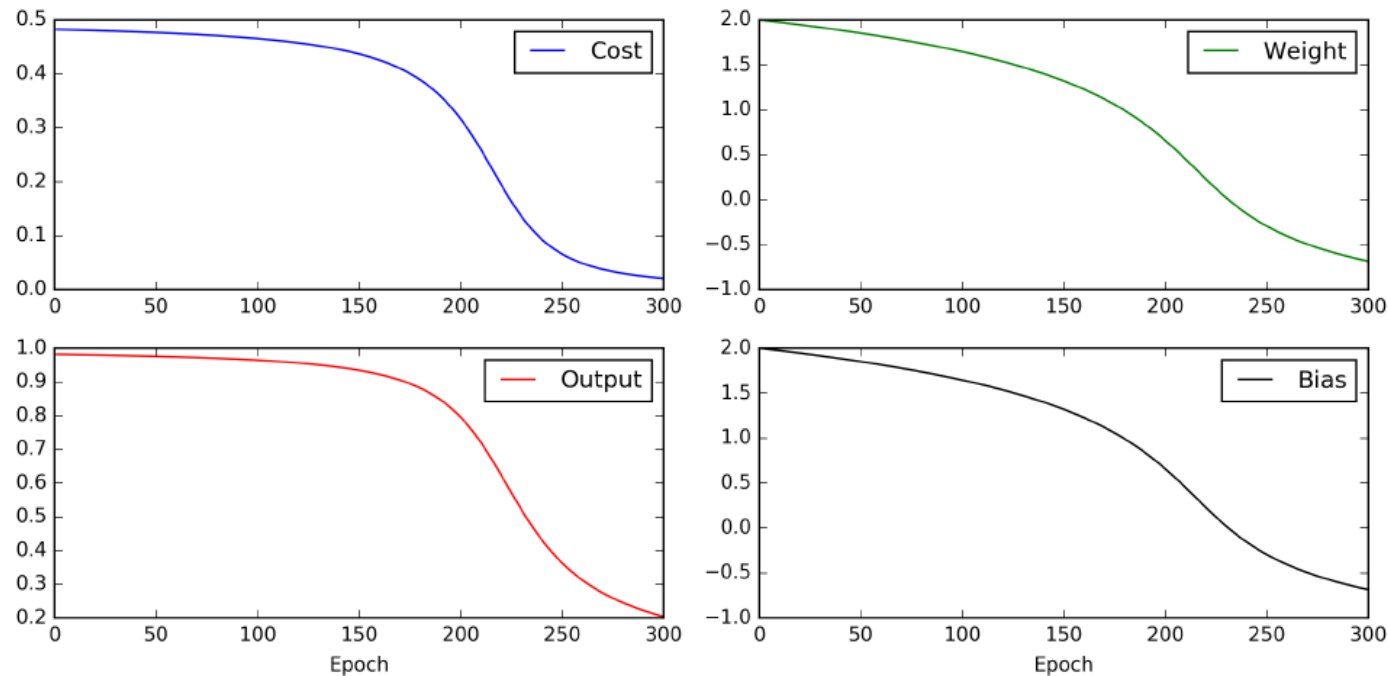
Cross-entropy cost function

- It'll be good if the neural network learns fast from it's errors
 - A toy network which take one input "1" and the desired output is "0"
 - Initial weight is "2.0" and initial bias is "2.0"
 - $z = "4"$ and $a = "0.98"$
 - $\eta=0.15$
 - Cost function is quadratic cost function



Cross-entropy cost function

- It'll be good if the neural network learns fast from it's errors
 - A toy network which take one input "1" and the desired output is "0"
 - Initial weight is "2.0" and initial bias is "2.0"
 - $z = "4"$ and $a = "0.98"$
 - $\eta = 0.15$
 - Cost function is quadratic cost function



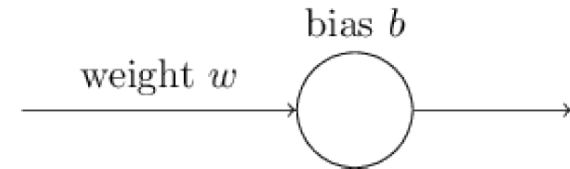
Cross-entropy cost function

- It'll be good if the neural network learns fast from it's errors
 - A toy network which take one input "1" and the desired output is "0"
 - Initial weight is "2.0" and initial bias is "2.0"
 - $z = "4"$ and $a = "0.98"$
 - $\eta=0.15$
 - Cost function is quadratic cost function

$$C = \frac{(y - a)^2}{2}$$

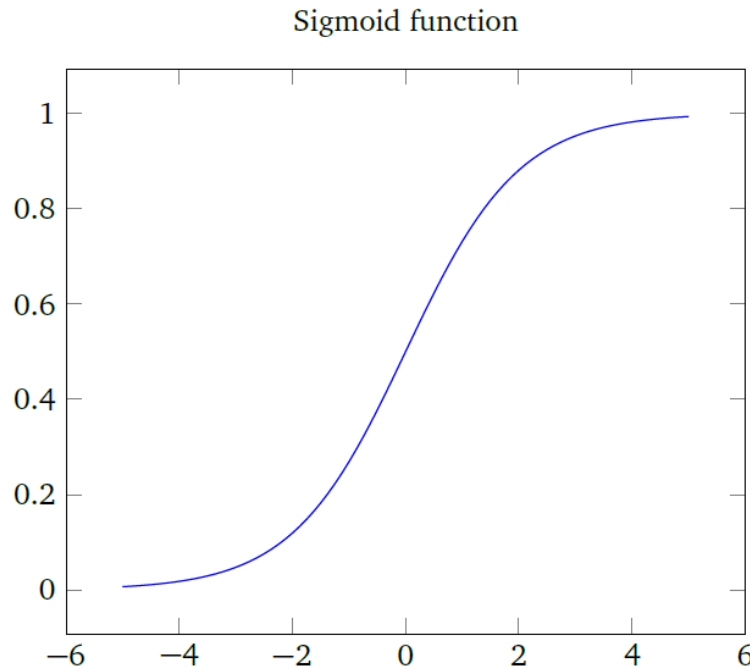
$$\frac{\partial C}{\partial w} = (a - y)\sigma'(z)x = a\sigma'(z)$$

$$\frac{\partial C}{\partial b} = (a - y)\sigma'(z) = a\sigma'(z),$$

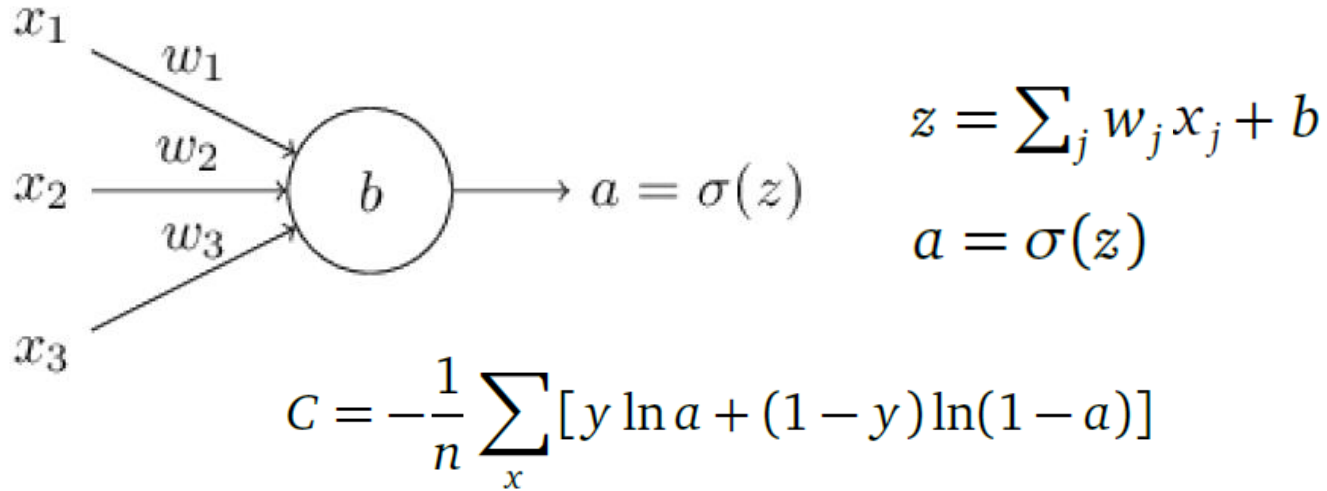


Cross-entropy cost function

- It'll be good if the neural network learns fast from it's errors
 - A toy network which take one input "1" and the desired output is "0"
 - Initial weight is "2.0" and initial bias is "2.0"
 - $z = "4"$ and $a = "0.98"$
 - $\eta = 0.15$
 - Cost function is quadratic cost function

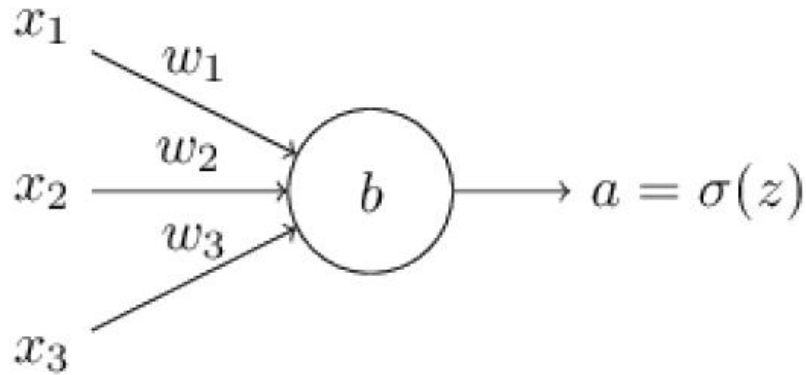


Cross-entropy cost function



Is this a Cost function?

Cross-entropy cost function



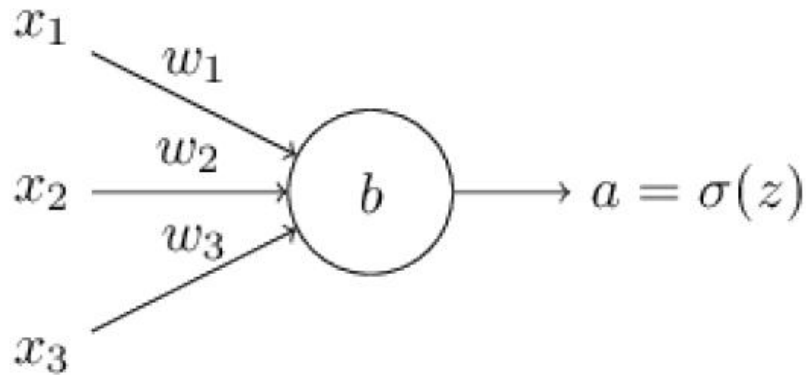
$$z = \sum_j w_j x_j + b$$

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$$

Does it fix the learning slowdown problem?

$$\frac{\partial C}{\partial w_j} =$$

Cross-entropy cost function



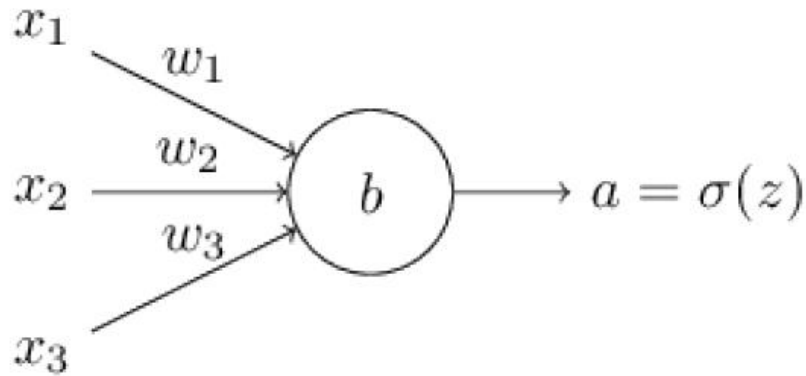
$$z = \sum_j w_j x_j + b$$

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$$

Does it fix the learning slowdown problem?

$$\frac{\partial C}{\partial w_j} = -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \frac{\partial \sigma}{\partial w_j}$$

Cross-entropy cost function



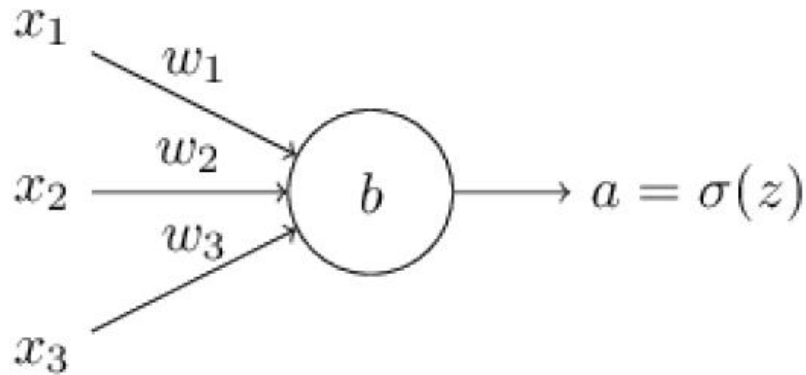
$$z = \sum_j w_j x_j + b$$

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$$

Does it fix the learning slowdown problem?

$$\begin{aligned} \frac{\partial C}{\partial w_j} &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \frac{\partial \sigma}{\partial w_j} \\ &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \sigma'(z) x_j \end{aligned}$$

Cross-entropy cost function



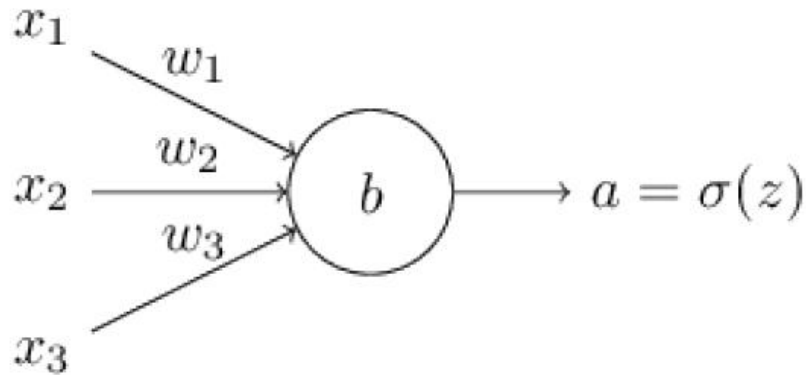
$$z = \sum_j w_j x_j + b$$

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$$

Does it fix the learning slowdown problem?

$$\begin{aligned} \frac{\partial C}{\partial w_j} &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \frac{\partial \sigma}{\partial w_j} \\ &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \sigma'(z) x_j \\ &= \frac{1}{n} \sum_x \frac{\sigma'(z) x_j}{\sigma(z)(1-\sigma(z))} (\sigma(z) - y) \end{aligned}$$

Cross-entropy cost function



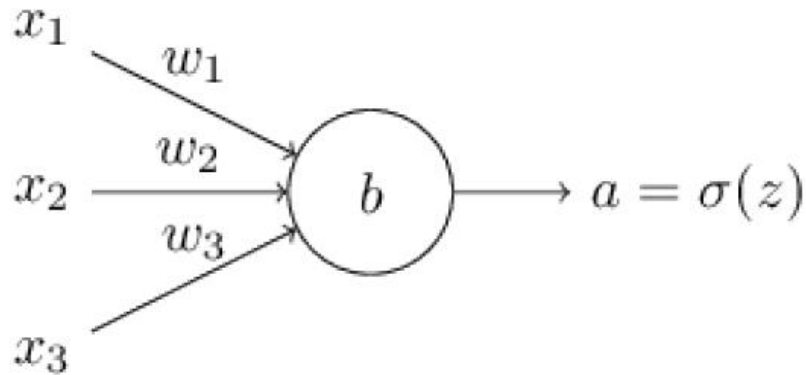
$$z = \sum_j w_j x_j + b$$

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$$

Does it fix the learning slowdown problem?

$$\begin{aligned} \frac{\partial C}{\partial w_j} &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \frac{\partial \sigma}{\partial w_j} \\ &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \sigma'(z) x_j \\ &= \frac{1}{n} \sum_x \frac{\sigma'(z) x_j}{\sigma(z)(1-\sigma(z))} (\sigma(z) - y) \\ \frac{\partial C}{\partial w_j} &= \frac{1}{n} \sum_x x_j (\sigma(z) - y) \end{aligned}$$

Cross-entropy cost function



$$z = \sum_j w_j x_j + b$$

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$$

Does it fix the learning slowdown problem?

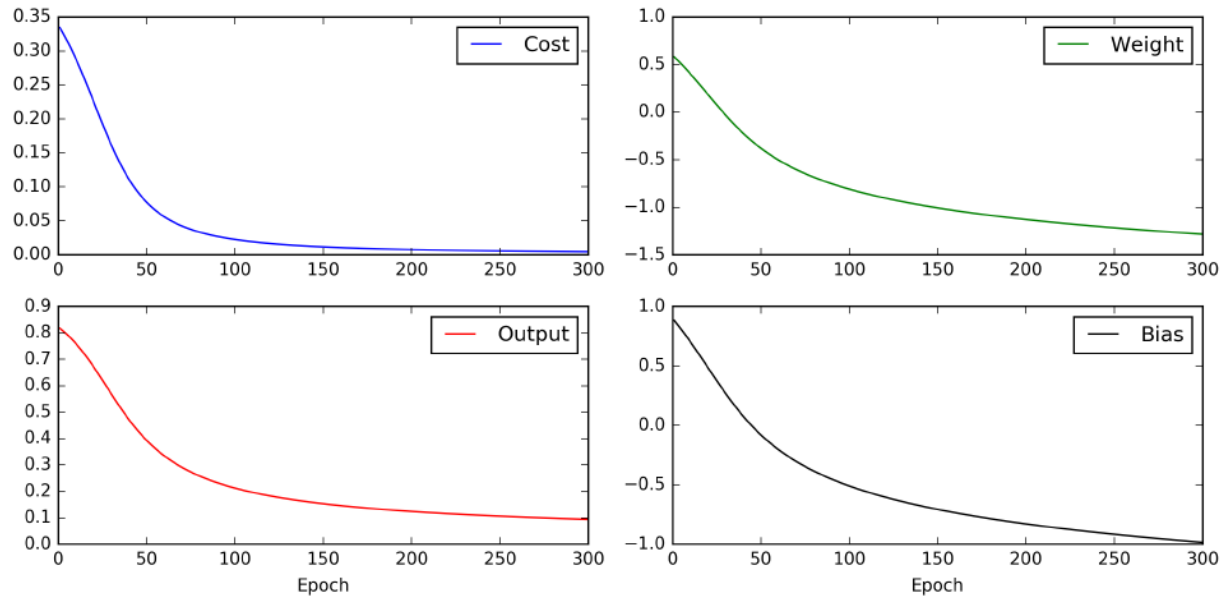
$$\begin{aligned} \frac{\partial C}{\partial w_j} &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \frac{\partial \sigma}{\partial w_j} \\ &= -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \sigma'(z) x_j \end{aligned}$$

$$= \frac{1}{n} \sum_x \frac{\sigma'(z) x_j}{\sigma(z)(1-\sigma(z))} (\sigma(z) - y)$$

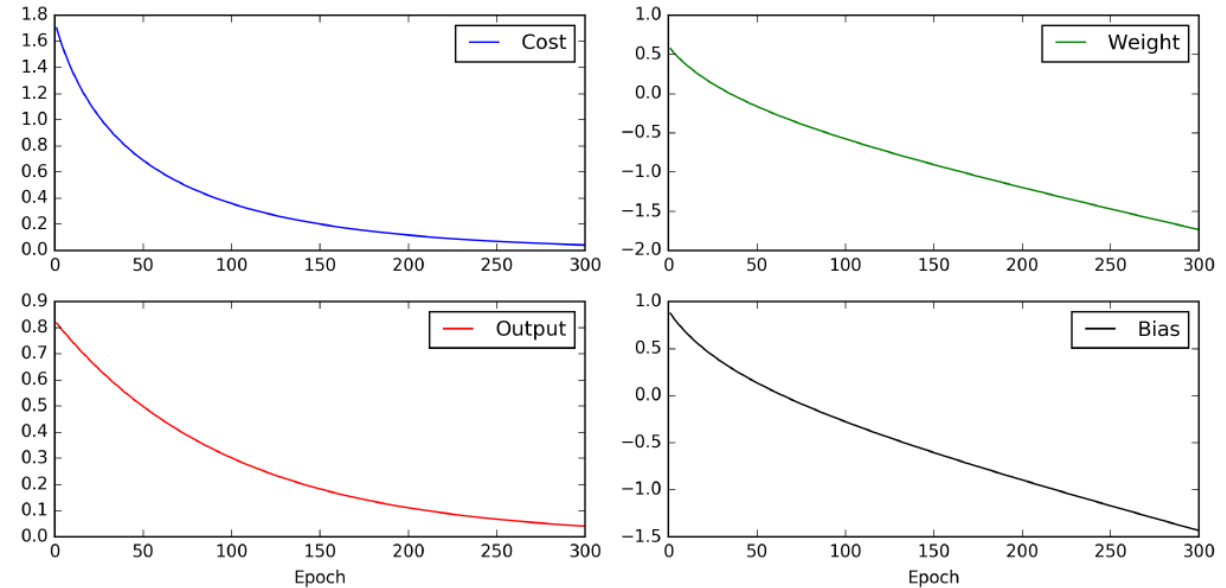
$$\frac{\partial C}{\partial w_j} = \frac{1}{n} \sum_x x_j (\sigma(z) - y) \quad \frac{\partial C}{\partial b} = \frac{1}{n} \sum_x (\sigma(z) - y)$$

Cross-entropy cost function

Quadratic cost function with $w=0.6$, $b=0.9$, $a=0.82$

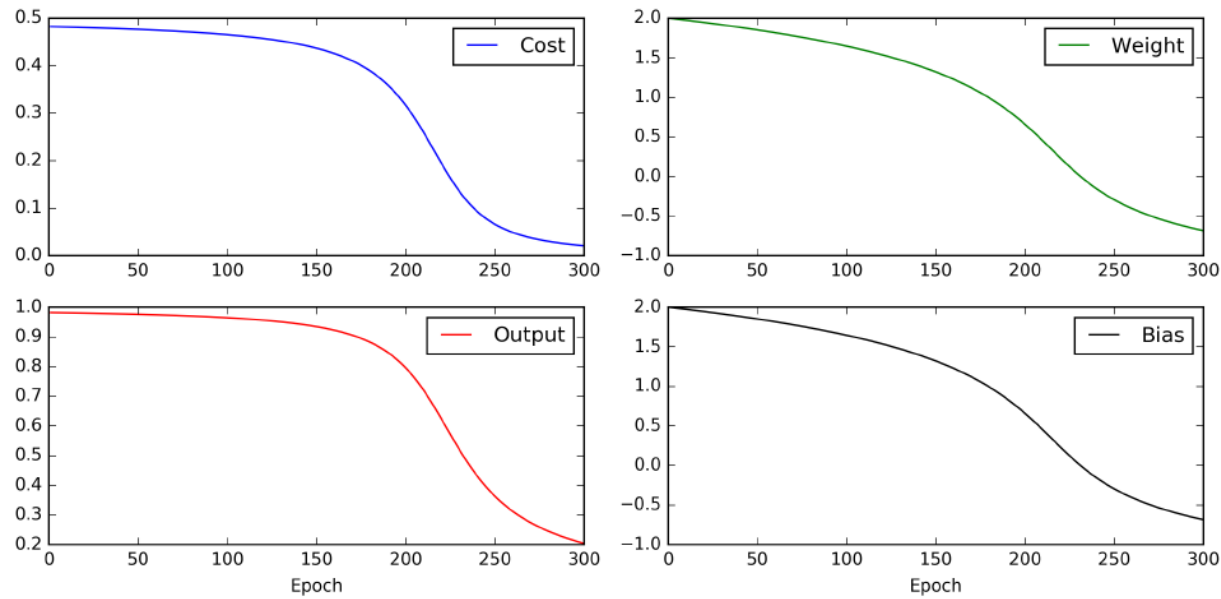


Cross-entropy cost function with $w=0.6$, $b=0.9$, $a=0.82$

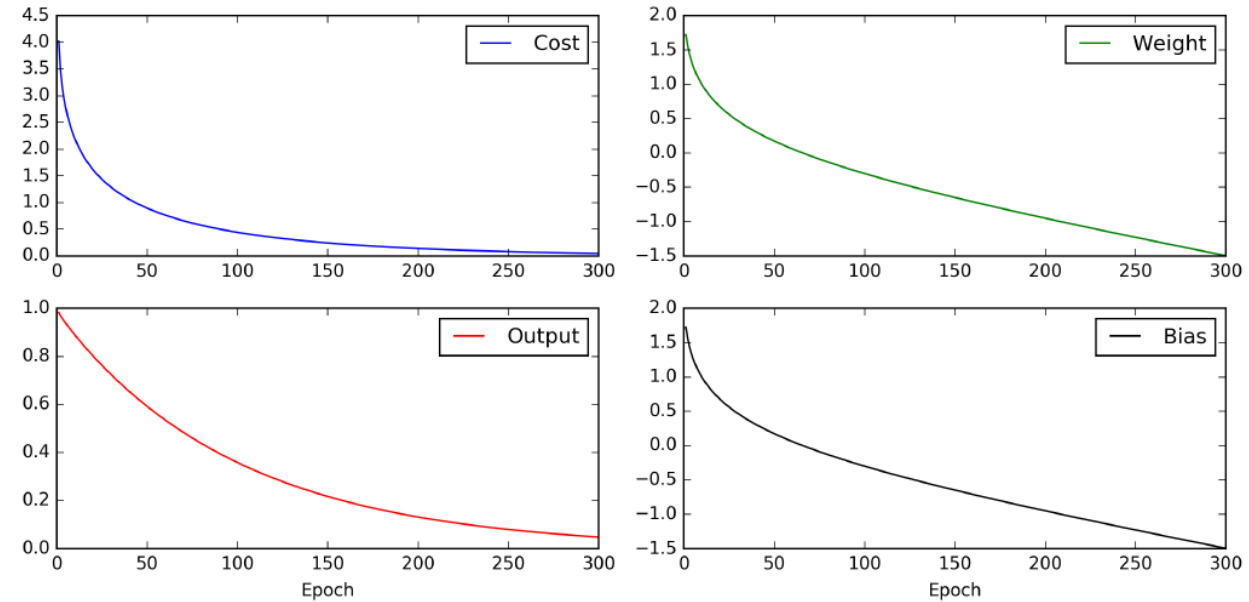


Cross-entropy cost function

Quadratic cost function with $w=b=2.0$, $a=0.98$



Cross-entropy cost function with $w=b=2.0$, $a=0.98$



Softmax

- Define a new type of output layer

$$a_j^L = \frac{e^{z_j^L}}{\sum_k e^{z_k^L}}$$

suppose we have a network with four output neurons,

and four corresponding weighted inputs z_1^L, z_2^L, z_3^L , and z_4^L

$$\sum_j a_j^L = \frac{\sum_j e^{z_j^L}}{\sum_k e^{z_k^L}} = 1$$

Softmax

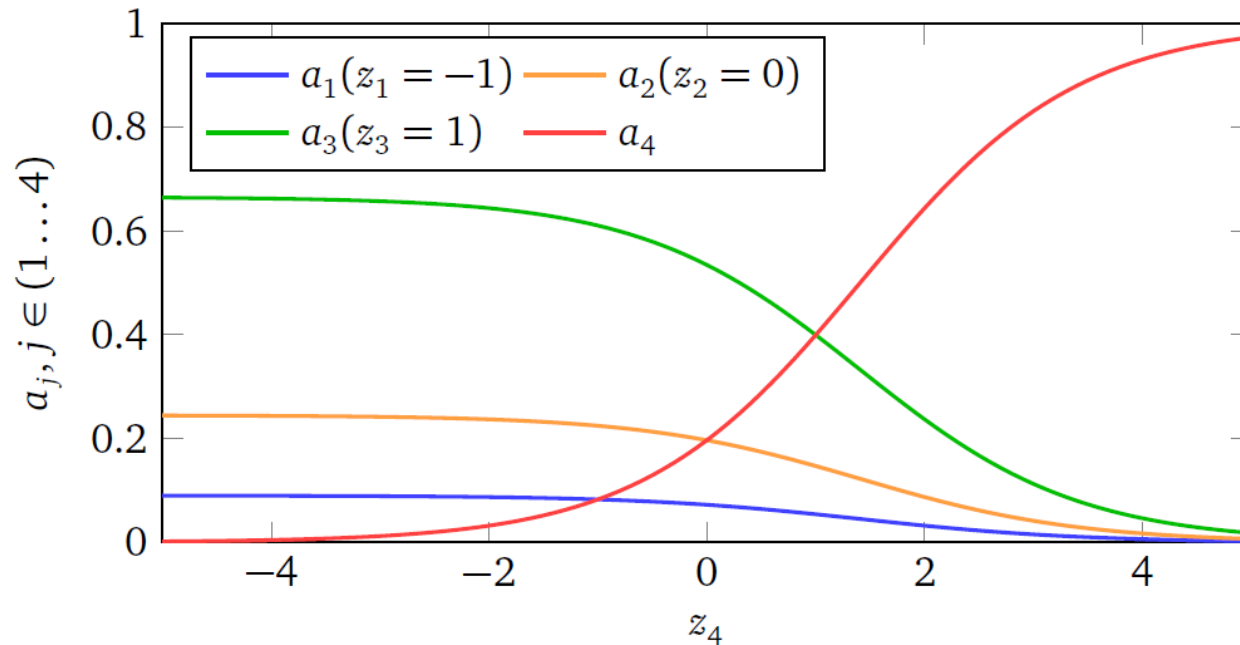
- Define a new type of output layer

$$a_j^L = \frac{e^{z_j^L}}{\sum_k e^{z_k^L}}$$

suppose we have a network with four output neurons,

and four corresponding weighted inputs z_1^L, z_2^L, z_3^L , and z_4^L

$$\sum_j a_j^L = \frac{\sum_j e^{z_j^L}}{\sum_k e^{z_k^L}} = 1$$

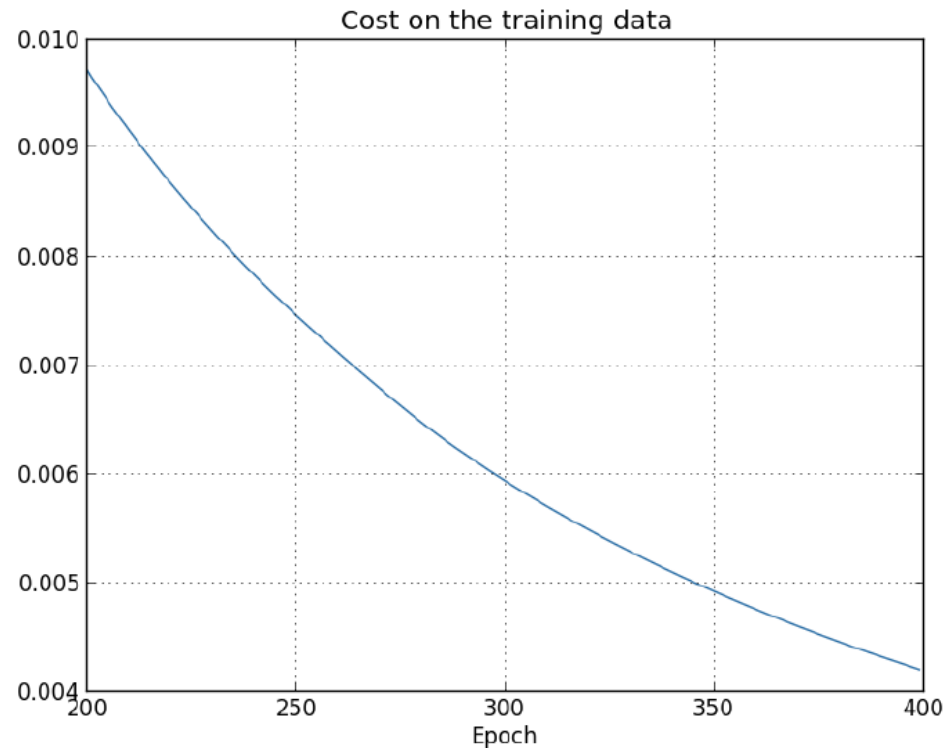


Overfitting and regularization

- New network of 30 hidden layer neurons
 - About 24000 variables
 - 1000 training images out of 50000 training images
 - Mini-batch size of 10 with $\eta = 0.5$
 - 400 epochs

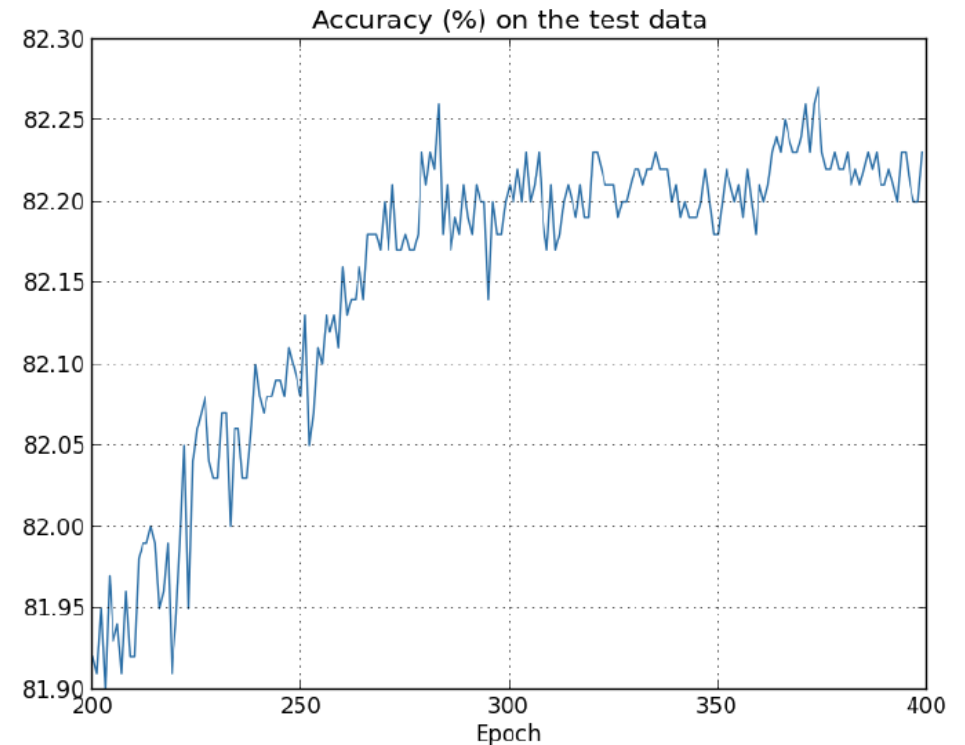
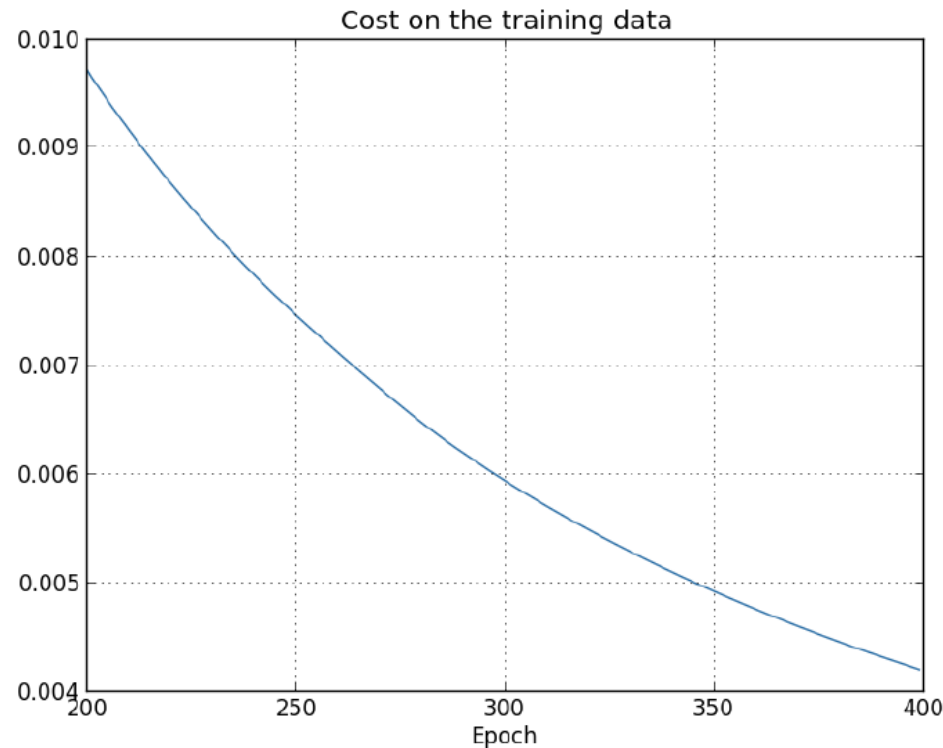
Overfitting and regularization

- New network of 30 hidden layer neurons
 - About 24000 variables
 - 1000 training images out of 50000 training images
 - Mini-batch size of 10 with $\eta = 0.5$
 - 400 epochs



Overfitting and regularization

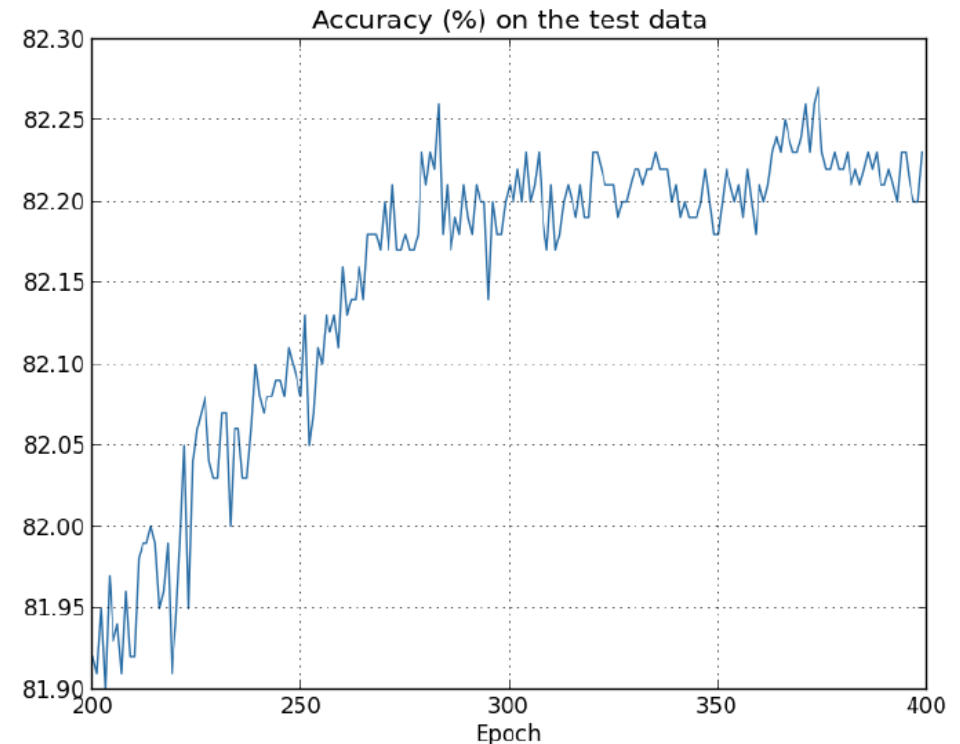
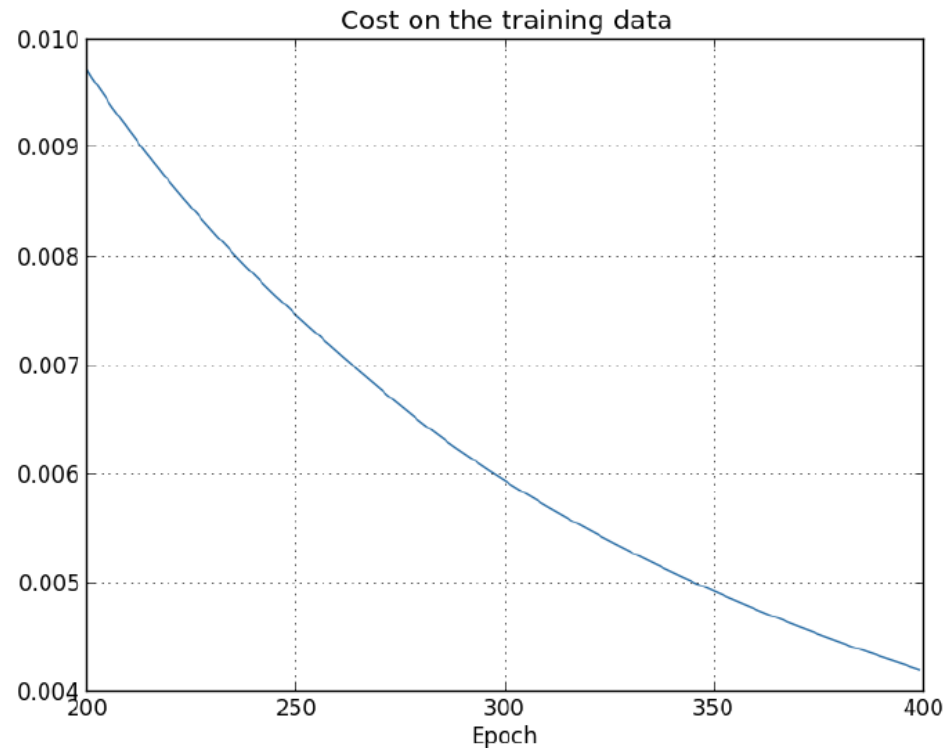
- New network of 30 hidden layer neurons
 - About 24000 variables
 - 1000 training images out of 50000 training images
 - Mini-batch size of 10 with $\eta = 0.5$
 - 400 epochs



Overfitting and regularization

- New network of 30 hidden layer neurons
 - About 24000 variables
 - 1000 training images out of 50000 training images
 - Mini-batch size of 10 with $\eta = 0.5$
 - 400 epochs

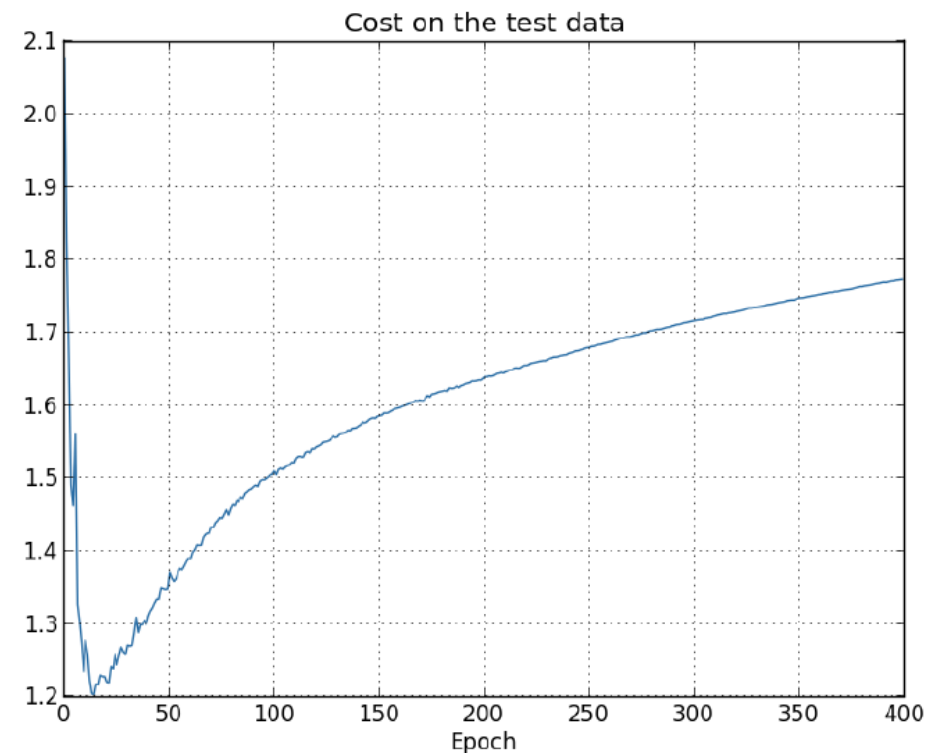
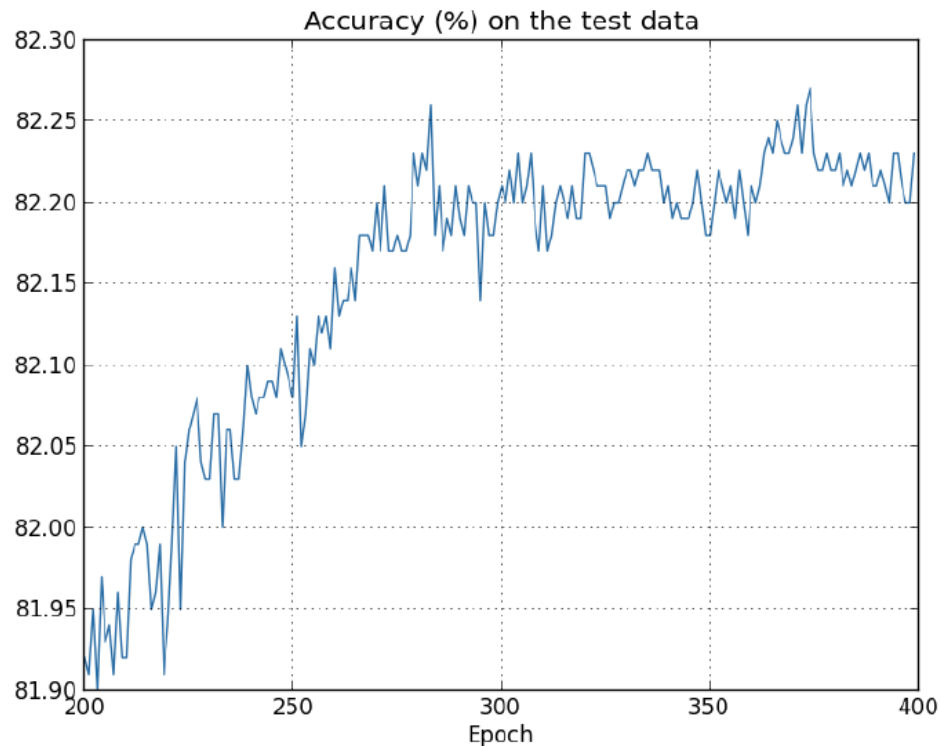
the network is *overfitting* or *overtraining* beyond epoch 280



Overfitting and regularization

- New network of 30 hidden layer neurons
 - About 24000 variables
 - 1000 training images out of 50000 training images
 - Mini-batch size of 10 with $\eta = 0.5$
 - 400 epochs

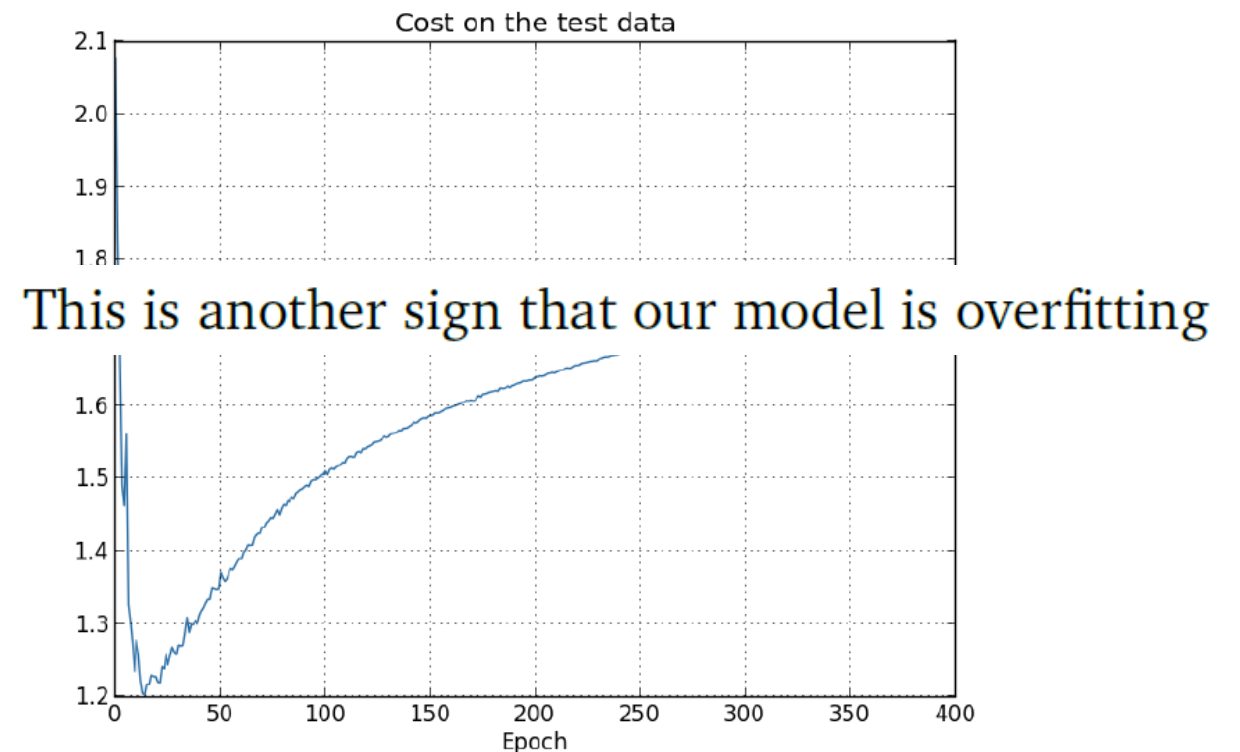
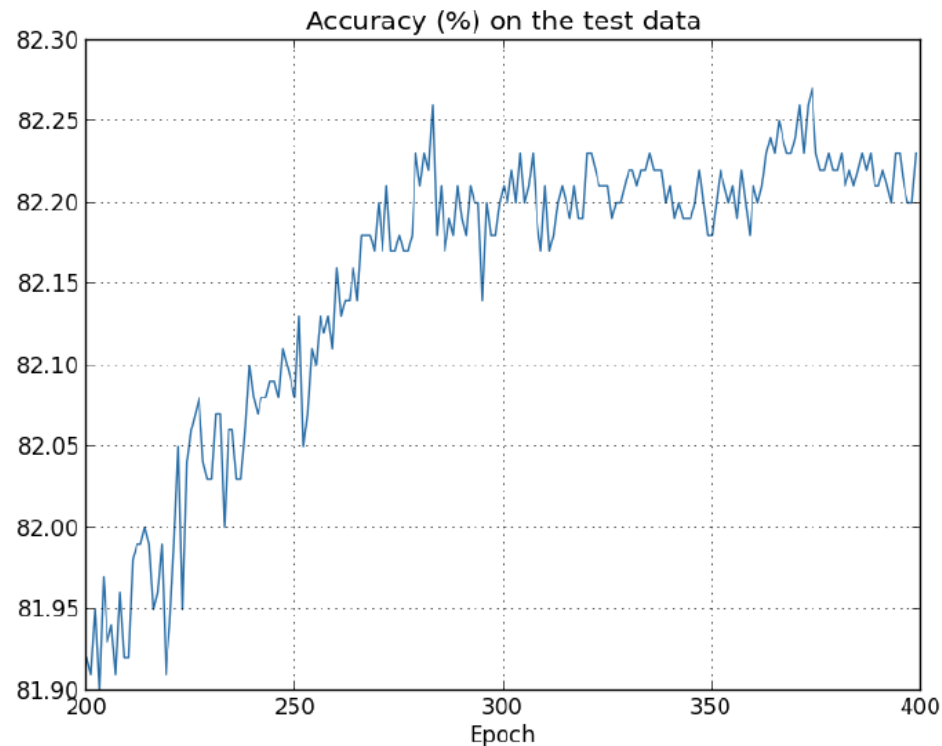
the network is *overfitting* or *overtraining* beyond epoch 280



Overfitting and regularization

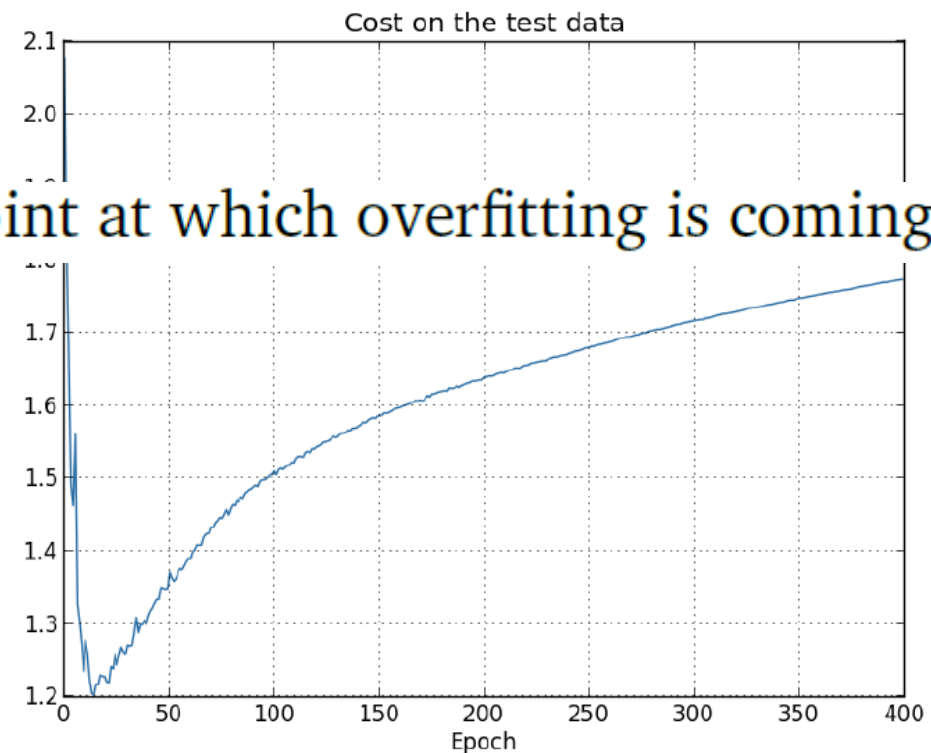
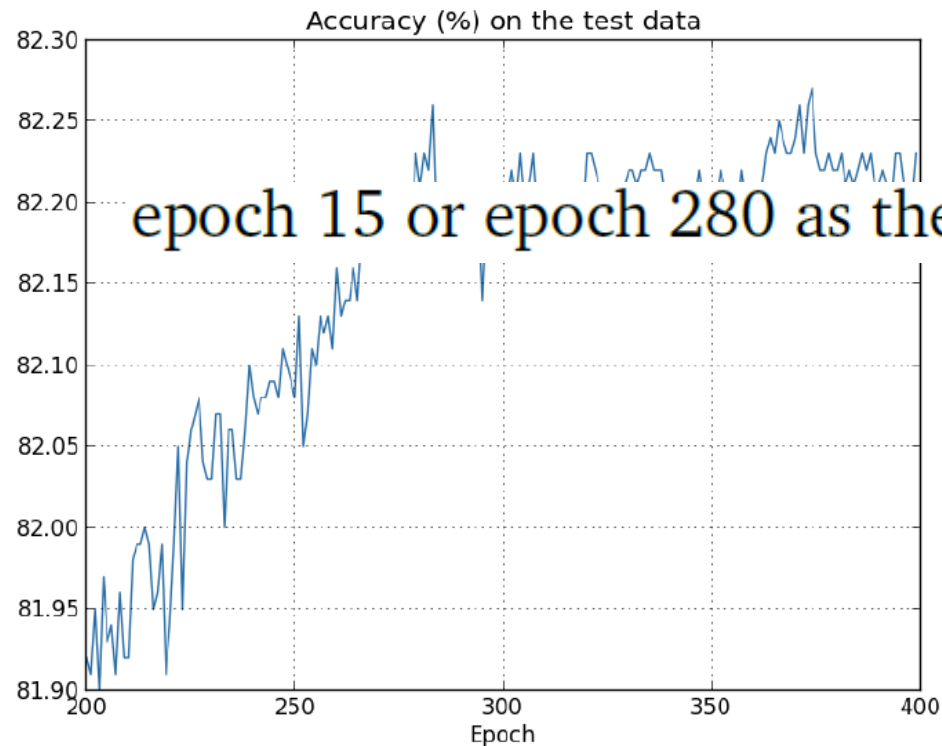
- New network of 30 hidden layer neurons
 - About 24000 variables
 - 1000 training images out of 50000 training images
 - Mini-batch size of 10 with $\eta = 0.5$
 - 400 epochs

the network is *overfitting* or *overtraining* beyond epoch 280

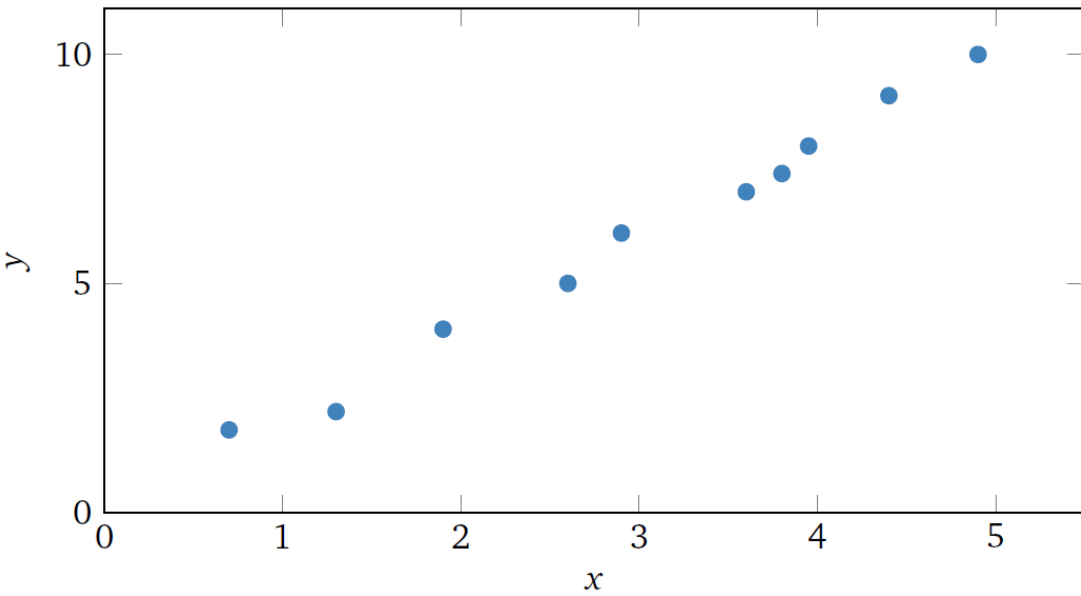


Overfitting and regularization

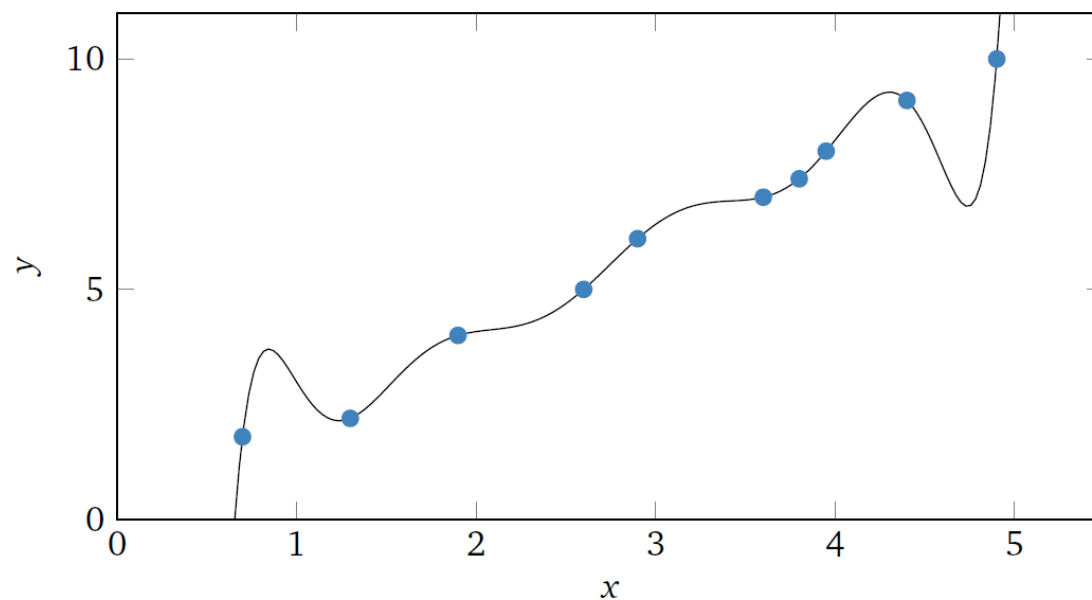
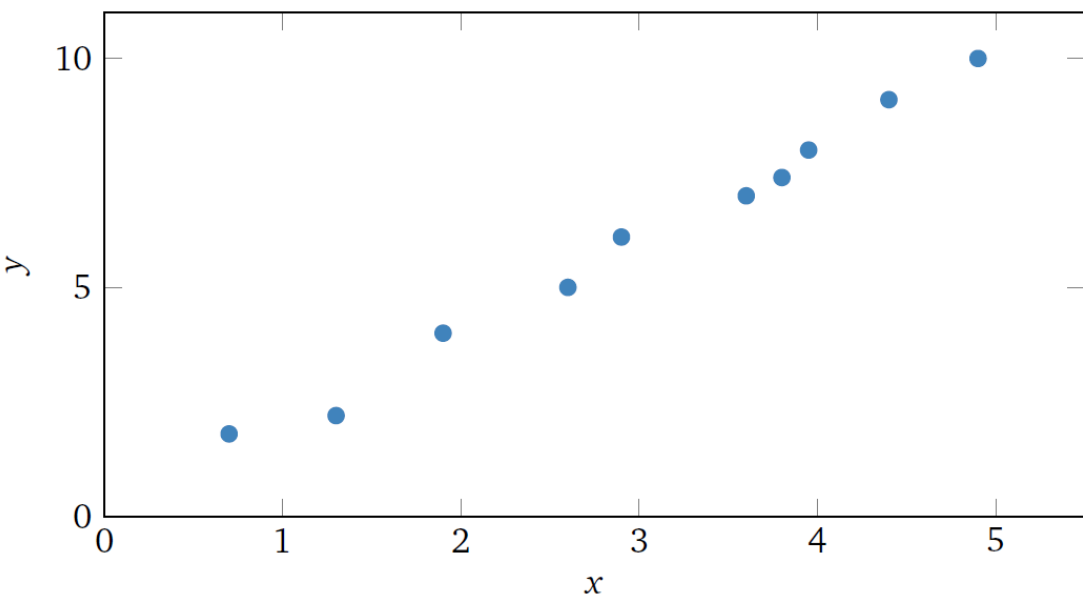
- New network of 30 hidden layer neurons
 - About 24000 variables
 - 1000 training images out of 50000 training images
 - Mini-batch size of 10 with $\eta = 0.5$
 - 400 epochs



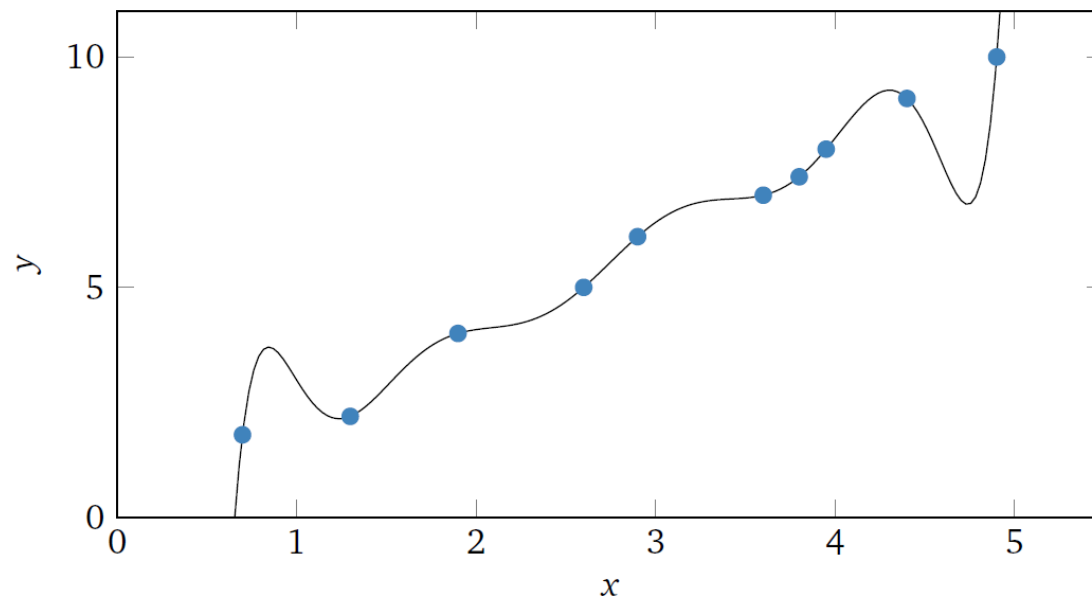
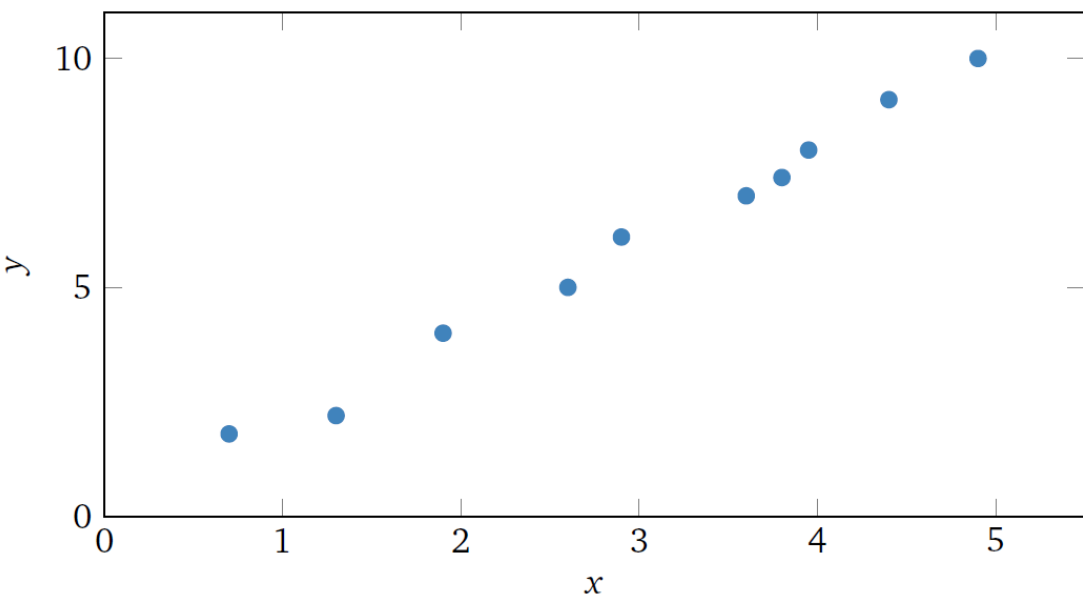
Overfitting?



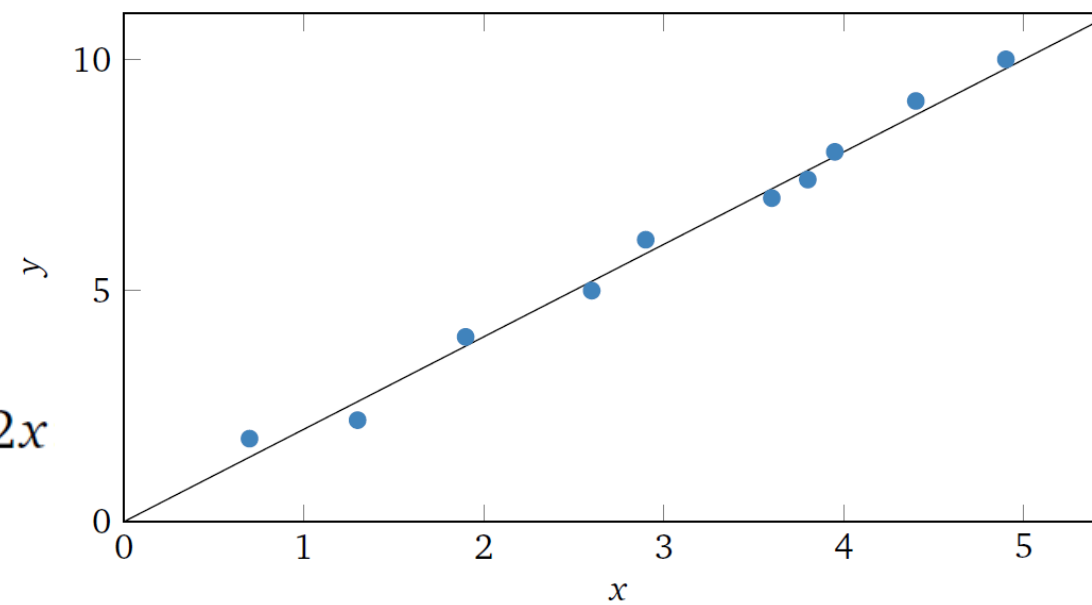
Overfitting?



Overfitting?



$$y = 2x$$



Regularization

- How to reduce overfitting?

Regularization

- How to reduce overfitting?
 - Increase the amount of training data

Regularization

- How to reduce overfitting?
 - Increase the amount of training data
 - Reduce the size of the network

Regularization

- How to reduce overfitting?
 - Increase the amount of training data
 - Reduce the size of the network
- Weight decay of L2 regularization
 - Add an extra term (regularization term) to the cost function

Regularization

- How to reduce overfitting?
 - Increase the amount of training data
 - Reduce the size of the network
- Weight decay of L2 regularization
 - Add an extra term (regularization term) to the cost function

$$C = \frac{1}{2n} \sum_x \|y - a^L\|^2 + \frac{\lambda}{2n} \sum_w w^2$$

Regularization

- How to reduce overfitting?
 - Increase the amount of training data
 - Reduce the size of the network
- Weight decay of L2 regularization
 - Add an extra term (regularization term) to the cost function

$$C = \frac{1}{2n} \sum_x \|y - a^L\|^2 + \frac{\lambda}{2n} \sum_w w^2$$

$$C = -\frac{1}{n} \sum_{xj} \left[y_j \ln a_j^L + (1 - y_j) \ln(1 - a_j^L) \right] + \frac{\lambda}{2n} \sum_w w^2$$

Regularization

- How to reduce overfitting?
 - Increase the amount of training data
 - Reduce the size of the network
- Weight decay of L2 regularization
 - Add an extra term (regularization term) to the cost function

$$C = \frac{1}{2n} \sum_x \|y - a^L\|^2 + \frac{\lambda}{2n} \sum_w w^2$$

$$C = -\frac{1}{n} \sum_{xj} \left[y_j \ln a_j^L + (1 - y_j) \ln(1 - a_j^L) \right] + \frac{\lambda}{2n} \sum_w w^2$$

$$C = C_0 + \frac{\lambda}{2n} \sum_w w^2$$

Regularization

- How to reduce overfitting?
 - Increase the amount of training data
 - Reduce the size of the network
- Weight decay of L2 regularization
 - Add an extra term (regularization term) to the cost function

$$\begin{aligned}\frac{\partial C}{\partial w} &= \frac{\partial C_0}{\partial w} + \frac{\lambda}{n} w \\ \frac{\partial C}{\partial b} &= \frac{\partial C_0}{\partial b}.\end{aligned}$$

$$C = \frac{1}{2n} \sum_x \|y - a^L\|^2 + \frac{\lambda}{2n} \sum_w w^2$$

$$C = -\frac{1}{n} \sum_{xj} \left[y_j \ln a_j^L + (1 - y_j) \ln(1 - a_j^L) \right] + \frac{\lambda}{2n} \sum_w w^2$$

$$C = C_0 + \frac{\lambda}{2n} \sum_w w^2$$

Regularization

- How to reduce overfitting?
 - Increase the amount of training data
 - Reduce the size of the network
- Weight decay of L2 regularization
 - Add an extra term (regularization term) to the cost function

$$\frac{\partial C}{\partial w} = \frac{\partial C_0}{\partial w} + \frac{\lambda}{n} w$$

$$\frac{\partial C}{\partial b} = \frac{\partial C_0}{\partial b}.$$

$$b \rightarrow b - \eta \frac{\partial C_0}{\partial b}$$

$$C = \frac{1}{2n} \sum_x \|y - a^L\|^2 + \frac{\lambda}{2n} \sum_w w^2 \quad w \rightarrow w - \eta \frac{\partial C_0}{\partial w} - \frac{\eta \lambda}{n} w = \left(1 - \frac{\eta \lambda}{n}\right) w - \eta \frac{\partial C_0}{\partial w}$$

$$C = -\frac{1}{n} \sum_{xj} \left[y_j \ln a_j^L + (1 - y_j) \ln(1 - a_j^L) \right] + \frac{\lambda}{2n} \sum_w w^2$$

$$C = C_0 + \frac{\lambda}{2n} \sum_w w^2$$

Regularization

- How to reduce overfitting?
 - Increase the amount of training data
 - Reduce the size of the network
- Weight decay of L2 regularization
 - Add an extra term (regularization term) to the cost function

$$\frac{\partial C}{\partial w} = \frac{\partial C_0}{\partial w} + \frac{\lambda}{n} w$$

$$\frac{\partial C}{\partial b} = \frac{\partial C_0}{\partial b}.$$

$$b \rightarrow b - \eta \frac{\partial C_0}{\partial b}$$

$$C = \frac{1}{2n} \sum_x \|y - a^L\|^2 + \frac{\lambda}{2n} \sum_w w^2 \quad w \rightarrow w - \eta \frac{\partial C_0}{\partial w} - \frac{\eta \lambda}{n} w = \left(1 - \frac{\eta \lambda}{n}\right) w - \eta \frac{\partial C_0}{\partial w}$$

$$C = -\frac{1}{n} \sum_{xj} \left[y_j \ln a_j^L + (1 - y_j) \ln(1 - a_j^L) \right] + \frac{\lambda}{2n} \sum_w w^2$$

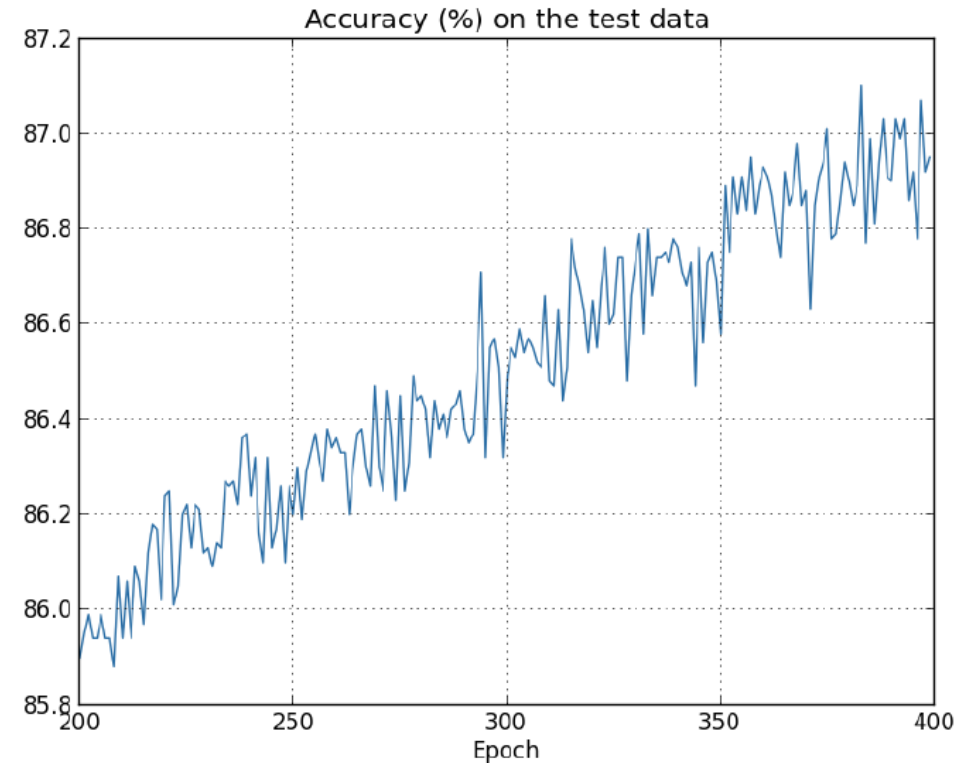
$$w \rightarrow \left(1 - \frac{\eta \lambda}{n}\right) w - \frac{\eta}{m} \sum_x \frac{\partial C_x}{\partial w}$$

$$C = C_0 + \frac{\lambda}{2n} \sum_w w^2$$

$$b \rightarrow b - \frac{\eta}{m} \sum_x \frac{\partial C_x}{\partial b}$$

Regularization

- New network of 30 hidden layer neurons
 - About 24000 variables
 - 1000 training images out of 50000 training images
 - Mini-batch size of 10 with $\eta = 0.5$ $\lambda = 0.1$
 - 400 epochs



Other techniques for regularization

- L1 regularization
- Dropout
- Artificial increase of training set size

L1 regularization

- The behavior of L1 regularization is different from the behavior of L2 regularization

$$C = C_0 + \frac{\lambda}{n} \sum_w |w|$$

L1 regularization

- The behavior of L1 regularization is different from the behavior of L2 regularization

$$C = C_0 + \frac{\lambda}{n} \sum_w |w|$$

$$\frac{\partial C}{\partial w} = \frac{\partial C_0}{\partial w} + \frac{\lambda}{n} \text{sgn}(w).$$

L1 regularization

- The behavior of L1 regularization is different from the behavior of L2 regularization

$$C = C_0 + \frac{\lambda}{n} \sum_w |w|$$

$$\frac{\partial C}{\partial w} = \frac{\partial C_0}{\partial w} + \frac{\lambda}{n} \text{sgn}(w).$$

$$w \rightarrow w' = w - \frac{\eta \lambda}{n} \text{sgn}(w) - \eta \frac{\partial C_0}{\partial w}$$

L1 regularization

- The behavior of L1 regularization is different from the behavior of L2 regularization

$$C = C_0 + \frac{\lambda}{n} \sum_w |w|$$

$$\frac{\partial C}{\partial w} = \frac{\partial C_0}{\partial w} + \frac{\lambda}{n} \text{sgn}(w).$$

$$w \rightarrow w' = w - \frac{\eta \lambda}{n} \text{sgn}(w) - \eta \frac{\partial C_0}{\partial w} \qquad w \rightarrow w' = w \left(1 - \frac{\eta \lambda}{n} \right) - \eta \frac{\partial C_0}{\partial w}$$

L1 regularization

- The behavior of L1 regularization is different from the behavior of L2 regularization

$$C = C_0 + \frac{\lambda}{n} \sum_w |w|$$

$$\frac{\partial C}{\partial w} = \frac{\partial C_0}{\partial w} + \frac{\lambda}{n} \text{sgn}(w).$$

$$w \rightarrow w' = w - \frac{\eta \lambda}{n} \text{sgn}(w) - \eta \frac{\partial C_0}{\partial w} \qquad w \rightarrow w' = w \left(1 - \frac{\eta \lambda}{n} \right) - \eta \frac{\partial C_0}{\partial w}$$

- In L1 regularization, the weights shrink by a constant amount toward 0
- In L2 regularization, the weights shrink by an amount proportional to w

L1 regularization

- The behavior of L1 regularization is different from the behavior of L2 regularization

$$C = C_0 + \frac{\lambda}{n} \sum_w |w|$$

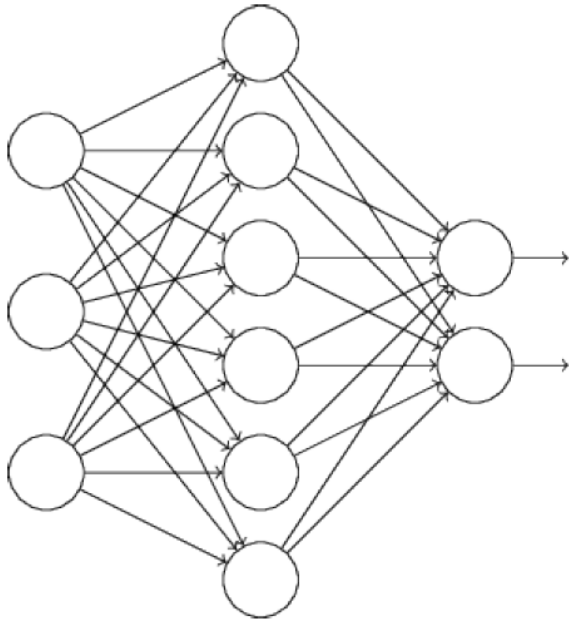
$$\frac{\partial C}{\partial w} = \frac{\partial C_0}{\partial w} + \frac{\lambda}{n} \text{sgn}(w).$$

$$w \rightarrow w' = w - \frac{\eta \lambda}{n} \text{sgn}(w) - \eta \frac{\partial C_0}{\partial w} \qquad w \rightarrow w' = w \left(1 - \frac{\eta \lambda}{n} \right) - \eta \frac{\partial C_0}{\partial w}$$

- In L1 regularization, the weights shrink by a constant amount toward 0
- In L2 regularization, the weights shrink by an amount proportional to w
- If a weight has a large magnitude, L2 regularization shrinks more weight
- If a weight has a small magnitude, L1 regularization shrinks more weight

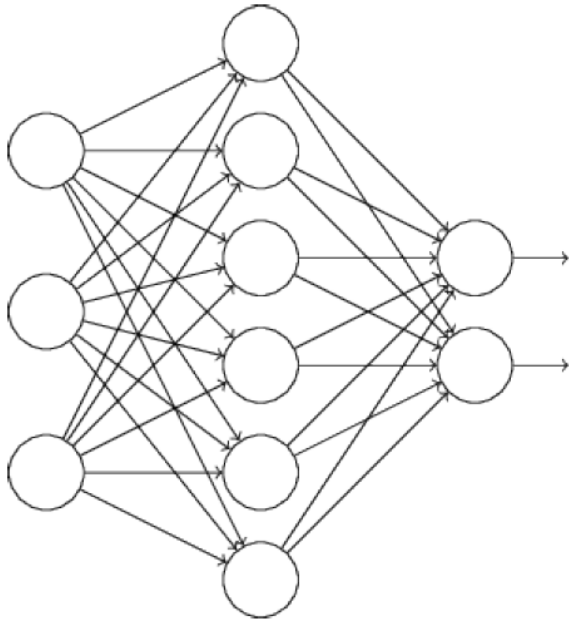
Dropout

- Try to modify the network
 - Not the cost function

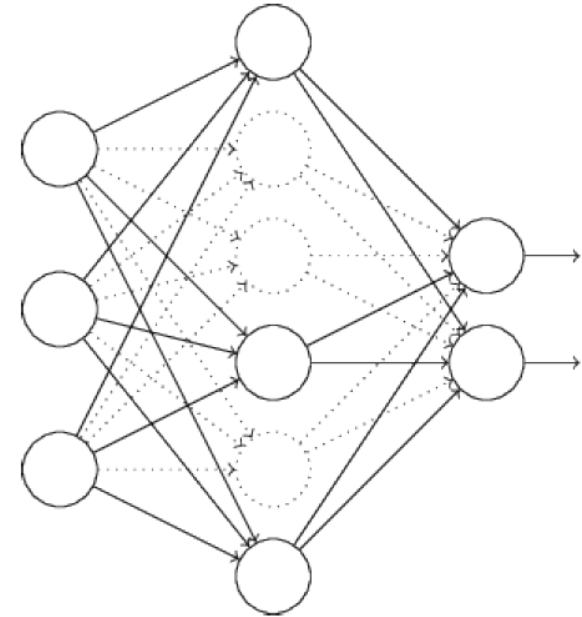


Dropout

- Try to modify the network
 - Not the cost function

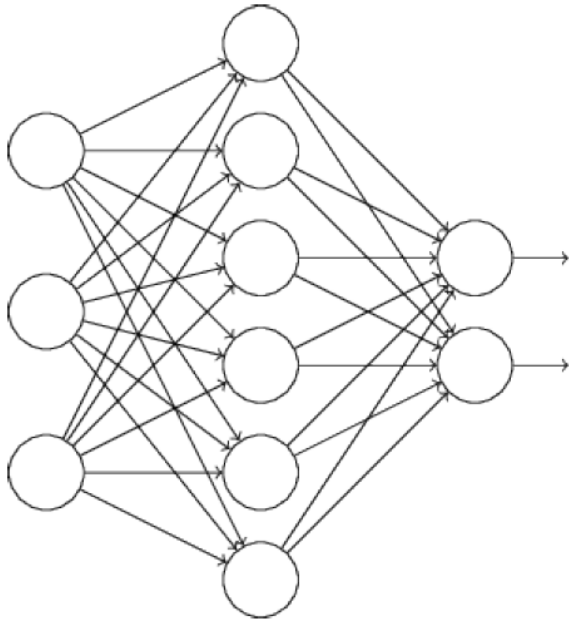


- Randomly dropout half of the hidden neurons
 - Feed forward
 - Back propagation

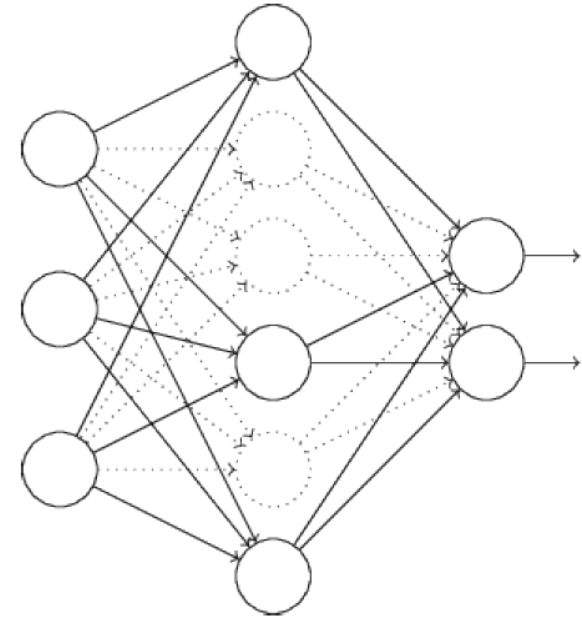


Dropout

- Try to modify the network
 - Not the cost function

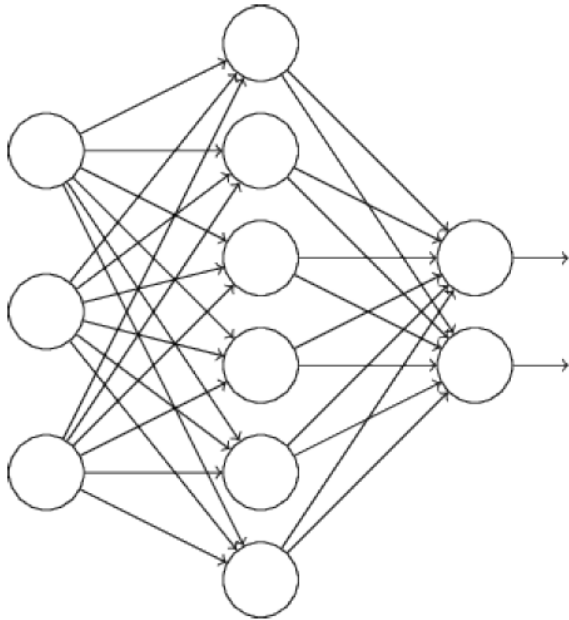


- Randomly dropout half of the hidden neurons
 - Feed forward
 - Back propagation
- Keep doing this during the mini-batches

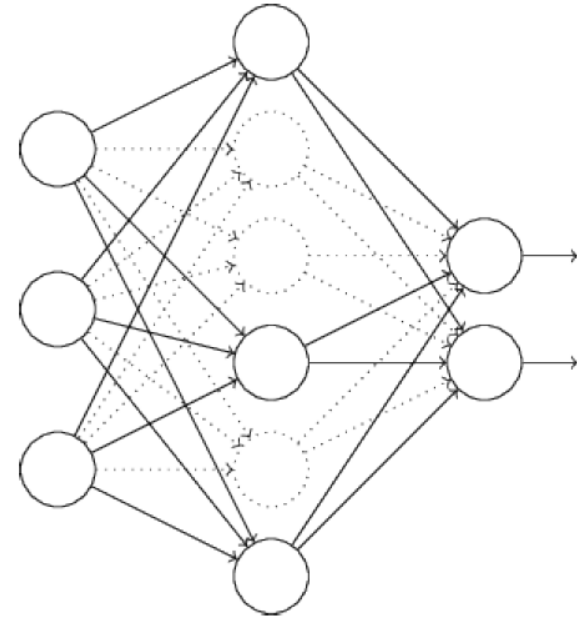


Dropout

- Try to modify the network
 - Not the cost function

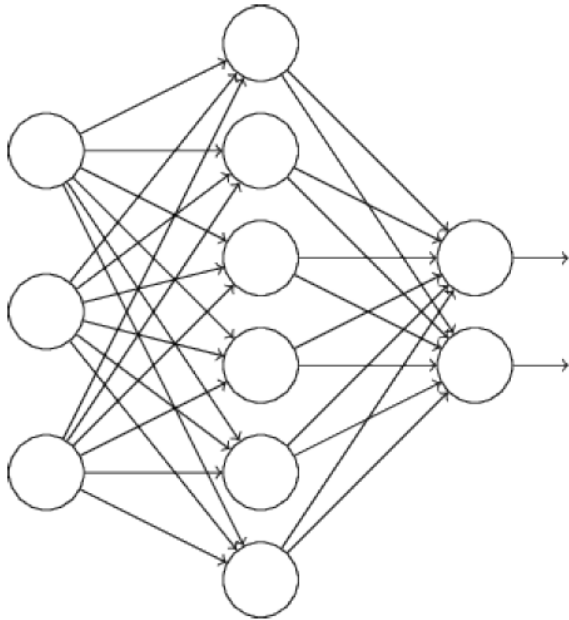


- Randomly dropout half of the hidden neurons
 - Feed forward
 - Back propagation
- Keep doing this during the mini-batches
- Restore all the hidden neurons after 1 epoch

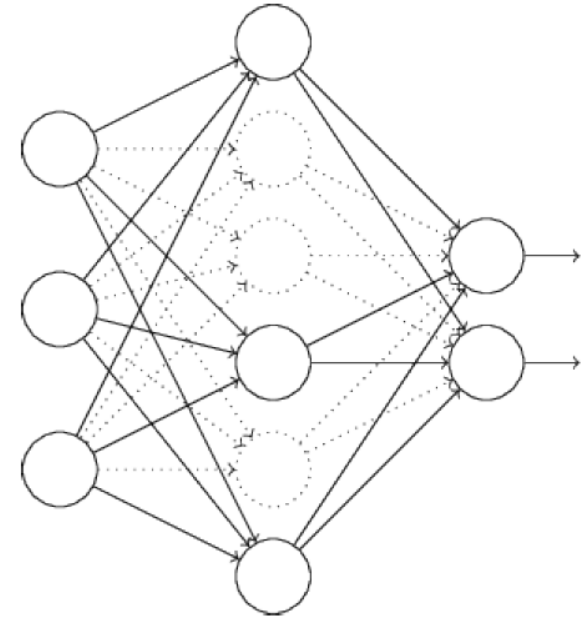


Dropout

- Try to modify the network
 - Not the cost function

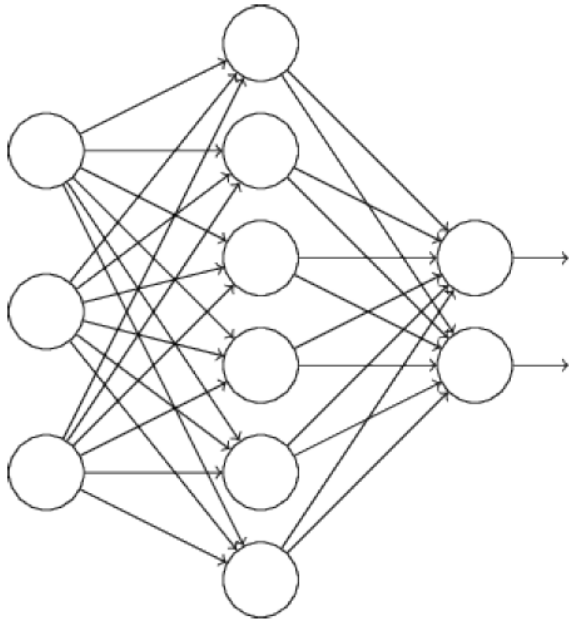


- Randomly dropout half of the hidden neurons
 - Feed forward
 - Back propagation
- Keep doing this during the mini-batches
- Restore all the hidden neurons after 1 epoch
 - Halve the outgoing weights from the hidden layer
 - Check the accuracy, cost, weights, biases of test data set

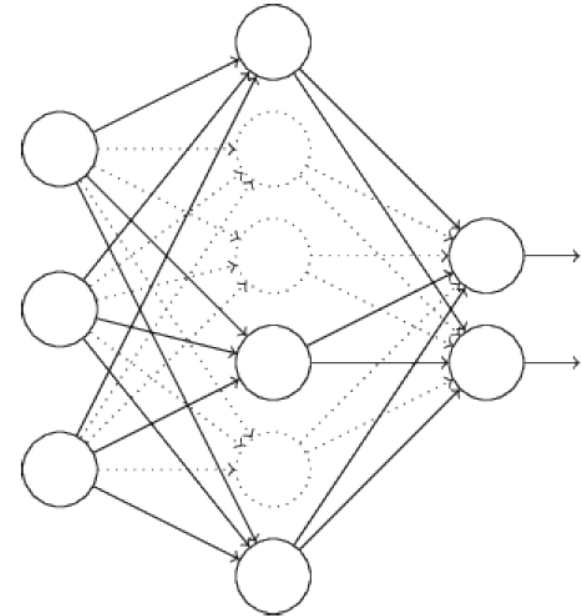


Dropout

- Try to modify the network
 - Not the cost function



- Randomly dropout half of the hidden neurons
 - Feed forward
 - Back propagation
- Keep doing this during the mini-batches
- Restore all the hidden neurons after 1 epoch
 - Halve the outgoing weights from the hidden layer
 - Check the accuracy, cost, weights, biases of test data set
- We are training different neural networks
 - Averaging effect



Artificially expanding training data

Artificially expanding training data

- Small rotation to the input images



rotate it by a small amount, let's say 15 degrees:

Artificially expanding training data

- Small rotation to the input images



rotate it by a small amount, let's say 15 degrees:



How to choose hyper-parameters

- Network design

How to choose hyper-parameters

- Network design
- Mini-batch size

How to choose hyper-parameters

- Network design
- Mini-batch size
- Number of epochs

How to choose hyper-parameters

- Network design
- Mini-batch size
- Number of epochs
- Learning rate η

How to choose hyper-parameters

- Network design
- Mini-batch size
- Number of epochs
- Learning rate η
- Regularization parameter λ

How to choose hyper-parameters

- Network design
- Mini-batch size
- Number of epochs
- Learning rate η
- Regularization parameter λ
- Cost function

How to choose hyper-parameters

- Network design
- Mini-batch size
- Number of epochs
- Learning rate η
- Regularization parameter λ
- Cost function
- Weight, bias initialization

How to choose hyper-parameters

- Network design
- Mini-batch size
- Number of epochs
- Learning rate η
- Regularization parameter λ
- Cost function
- Weight, bias initialization
- Activation function

Mini-batch size

- Large mini-batch size

Mini-batch size

- Large mini-batch size
 - Fast using linear algebra libraries

Mini-batch size

- Large mini-batch size
 - Fast using linear algebra libraries
 - Can't update weights and biases often

Mini-batch size

- Large mini-batch size
 - Fast using linear algebra libraries
 - Can't update weights and biases often
- Need compromise

Mini-batch size

- Large mini-batch size
 - Fast using linear algebra libraries
 - Can't update weights and biases often
- Need compromise
 - Plot "time" vs "accuracy" graph and choose the best mini-batch

Hyper parameters

- The number of epochs

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs
 - If classification doesn't improve for some time (ex, 10 epochs) stop and adjust hyper-parameters

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs
 - If classification doesn't improve for some time (ex, 10 epochs) stop and adjust hyper-parameters
- The learning rate

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs
 - If classification doesn't improve for some time (ex, 10 epochs) stop and adjust hyper-parameters
- The learning rate
 - Start with big learning rate

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs
 - If classification doesn't improve for some time (ex, 10 epochs) stop and adjust hyper-parameters
- The learning rate
 - Start with big learning rate
 - If accuracy gets worse, decrease learning rate η

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs
 - If classification doesn't improve for some time (ex, 10 epochs) stop and adjust hyper-parameters
- The learning rate
 - Start with big learning rate
 - If accuracy gets worse, decrease learning rate η
 - Similar to early stopping

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs
 - If classification doesn't improve for some time (ex, 10 epochs) stop and adjust hyper-parameters
- The learning rate
 - Start with big learning rate
 - If accuracy gets worse, decrease learning rate η
 - Similar to early stopping
- Regularization parameter

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs
 - If classification doesn't improve for some time (ex, 10 epochs) stop and adjust hyper-parameters
- The learning rate
 - Start with big learning rate
 - If accuracy gets worse, decrease learning rate η
 - Similar to early stopping
- Regularization parameter
 - Start with $\lambda_1 = 0$ and find η first

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs
 - If classification doesn't improve for some time (ex, 10 epochs) stop and adjust hyper-parameters
- The learning rate
 - Start with big learning rate
 - If accuracy gets worse, decrease learning rate η
 - Similar to early stopping
- Regularization parameter
 - Start with $\lambda_i = 0$ and find η first
 - Then $\lambda_i = 1$ and redo η

Hyper parameters

- The number of epochs
 - Use early stopping to determine the number of epochs
 - If classification doesn't improve for some time (ex, 10 epochs) stop and adjust hyper-parameters
- The learning rate
 - Start with big learning rate
 - If accuracy gets worse, decrease learning rate η
 - Similar to early stopping
- Regularization parameter
 - Start with $\lambda_i = 0$ and find η first
 - Then $\lambda_i = 1$ and redo η
 - Increase or decrease λ_i by a factor of 10 and redo

Other activation functions - tanh

$$\tanh(w \cdot x + b)$$

Other activation functions - tanh

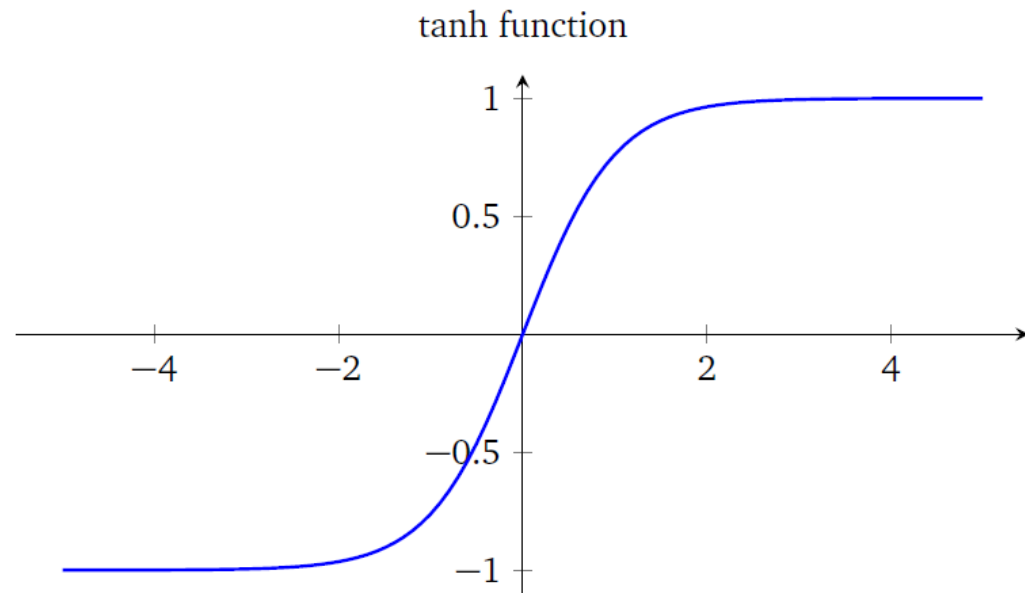
$$\tanh(w \cdot x + b)$$

$$\tanh(z) \equiv \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Other activation functions - tanh

$$\tanh(w \cdot x + b)$$

$$\tanh(z) \equiv \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

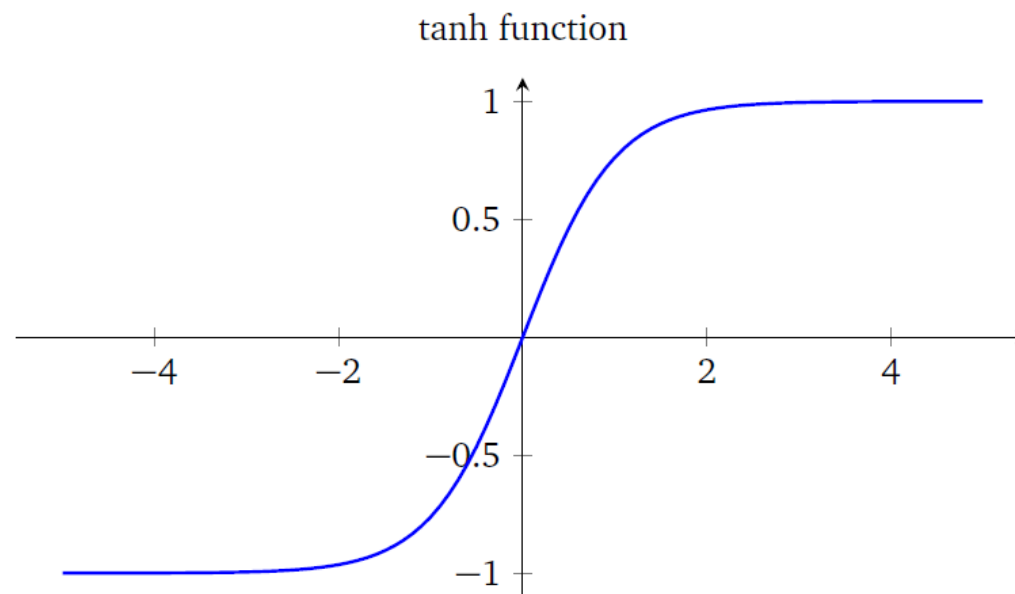


Other activation functions - tanh

$$\tanh(w \cdot x + b)$$

$$\tanh(z) \equiv \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\sigma(z) = \frac{1 + \tanh(z/2)}{2}$$

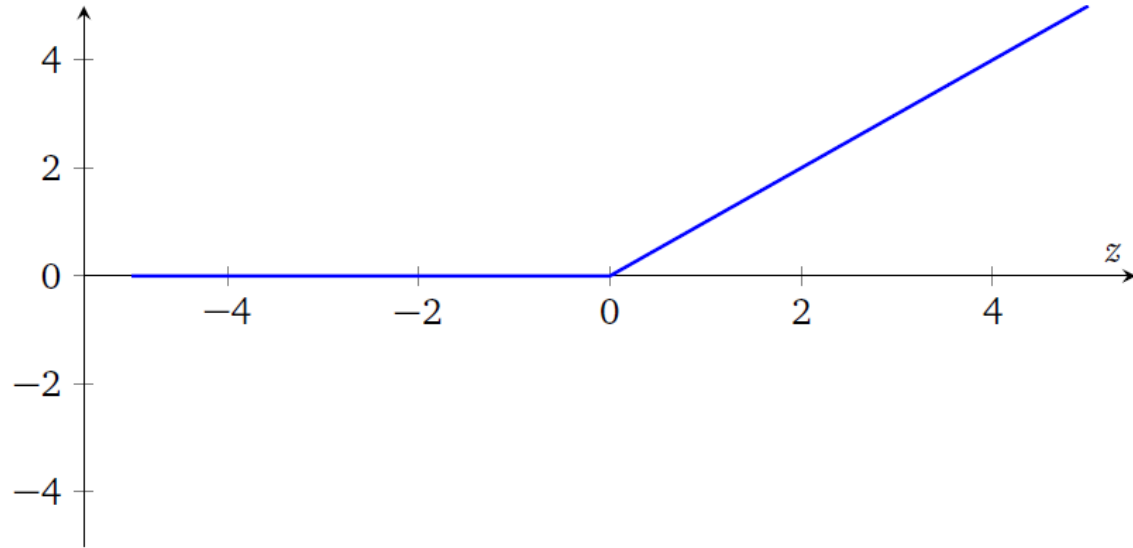


Other activation functions – rectified linear neuron

$$\max(0, w \cdot x + b).$$

Other activation functions – rectified linear neuron

$$\max(0, w \cdot x + b).$$



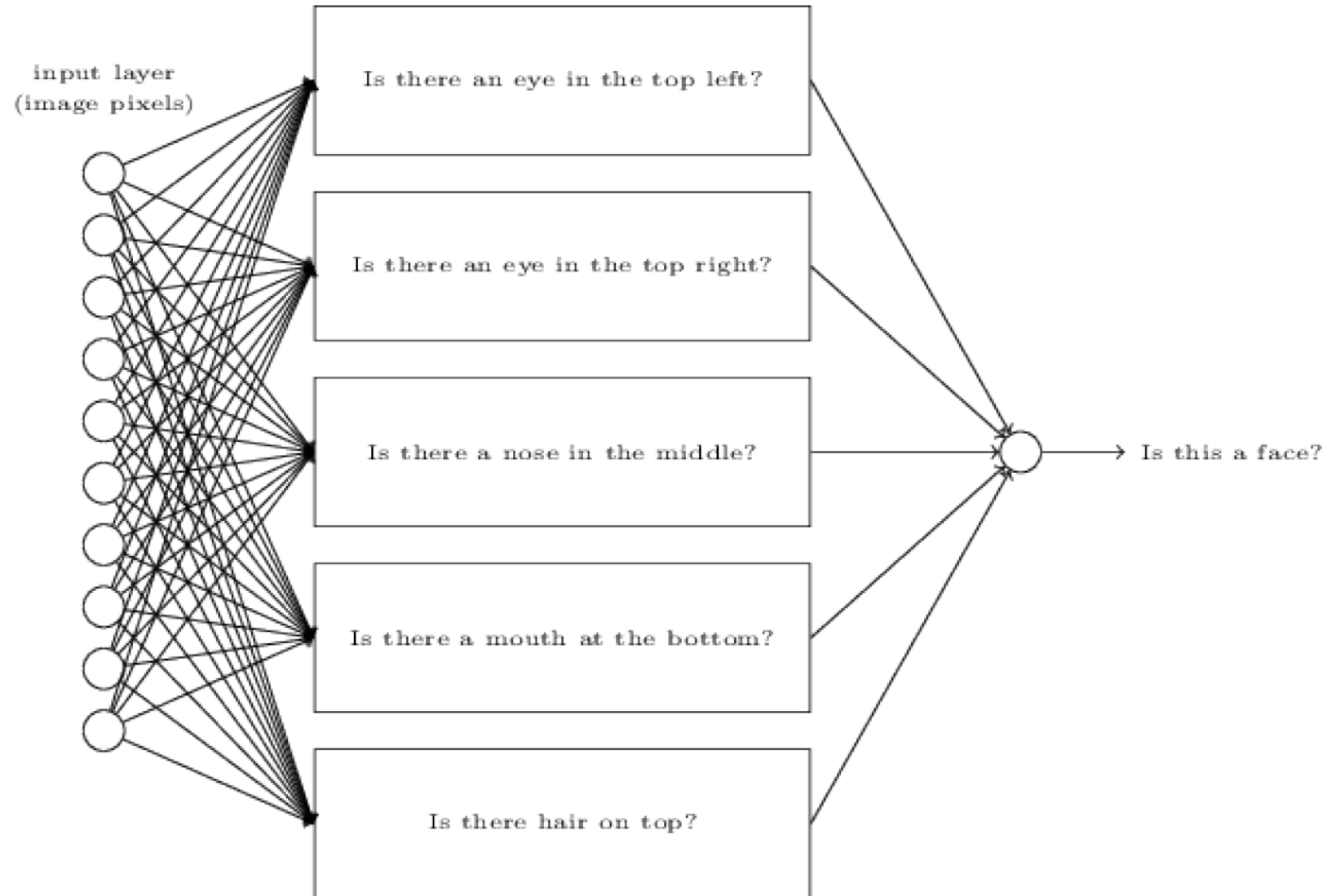
Toward Deep Learning

- **The weights and biases in the network are learned automatically**
 - **It's not easy to understand what each neuron does**
 - **Researchers have tried to understand the principles of how human brain and neural networks are working**
 - **We may not understand neither the brain nor the network works even in the future**

Toward Deep Learning

- **Think about a network recognizing human faces**
 - **Input neurons take pixels as inputs**
 - **A single output neuron will indicate “Yes, it’s a face” or “No, it’s not a face”**
 - **Decompose a problem into subproblems**
 - **Does it have an eye in the top right?**
 - **Does it have a nose in the middle?**
 - **Does it have a mouth in the bottom middle?**
 - **Is there hair on top?**
 - **If the answers to above questions are “yes” or just “probably yes” then the output neuron will indicate “Yes, it’s a face”**

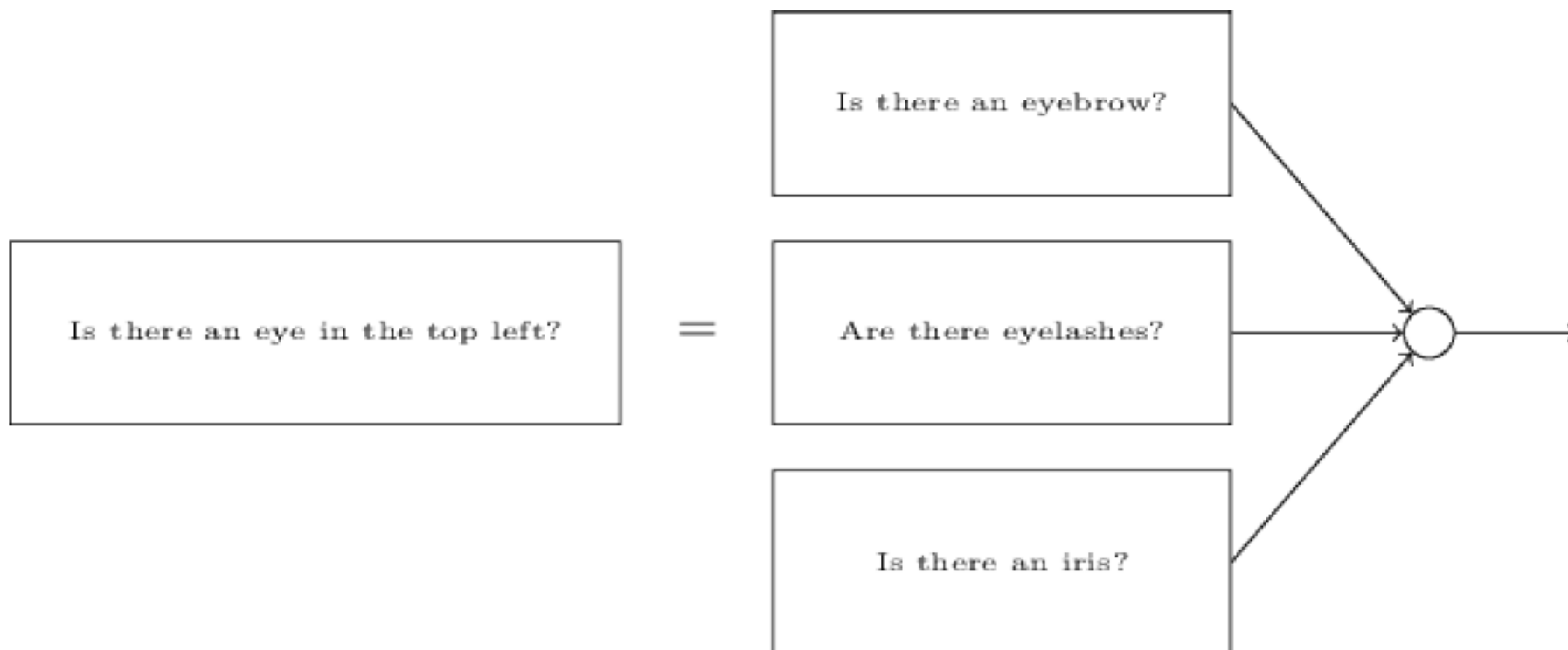
Toward Deep Learning



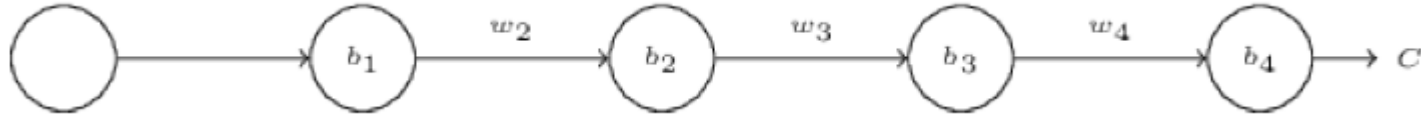
Toward Deep Learning

- **“Is there an eye in the top left?” can be decomposed into questions such as**
 - **“Is there an eyebrow?”**
 - **“Are there eyelashes?”**
 - **“Is there an iris?”**

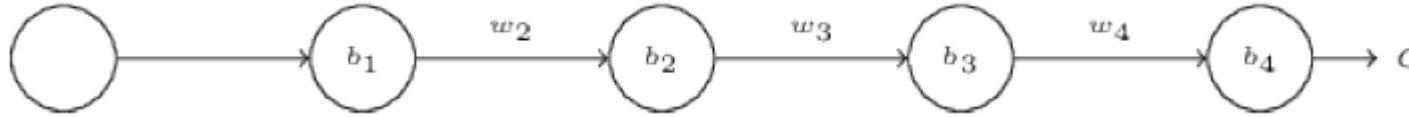
Toward Deep Learning



Vanishing gradient problems in deep learning



Vanishing gradient problems in deep learning



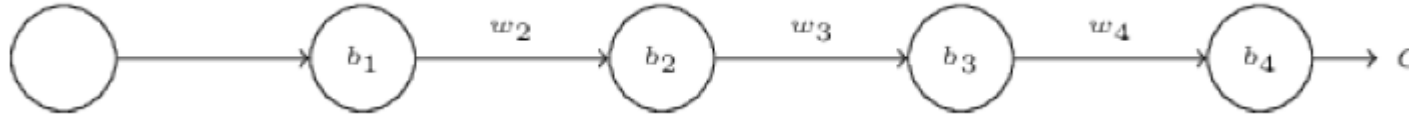
$$\delta^L = \nabla_a C \odot \sigma'(z^L) \quad (\text{BP1})$$

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l) \quad (\text{BP2})$$

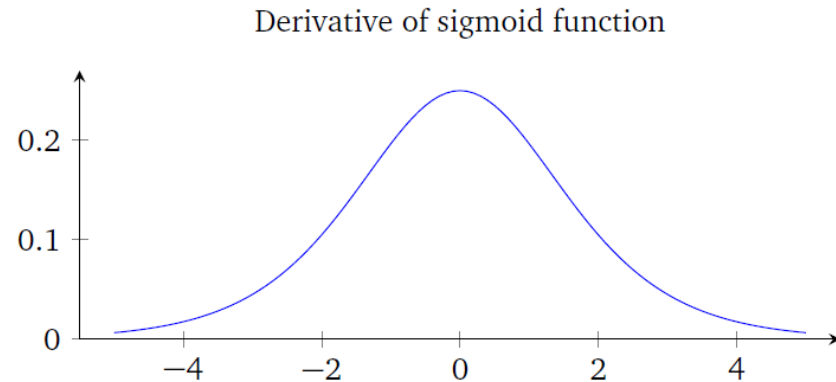
$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \quad (\text{BP3})$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \quad (\text{BP4})$$

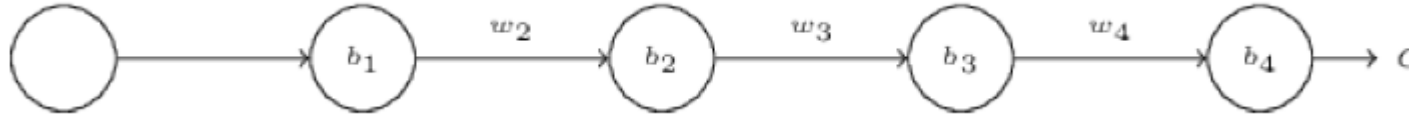
Vanishing gradient problems in deep learning



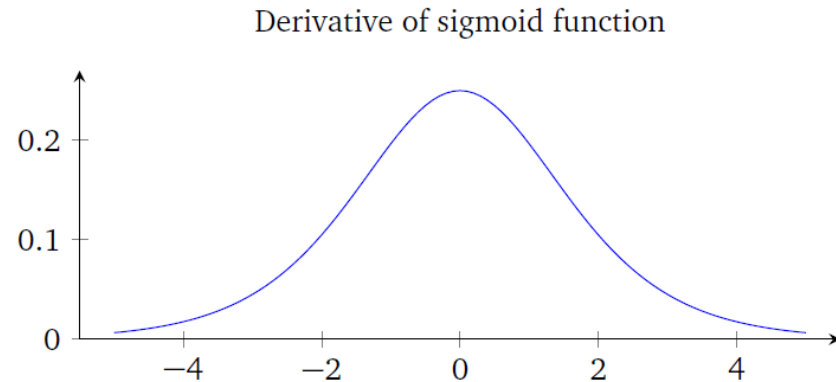
$$\frac{\partial C}{\partial b_1} = \sigma'(z_1) w_2 \sigma'(z_2) w_3 \sigma'(z_3) w_4 \sigma'(z_4) \frac{\partial C}{\partial a_4}$$



Vanishing gradient problems in deep learning



$$\frac{\partial C}{\partial b_1} = \sigma'(z_1) w_2 \sigma'(z_2) w_3 \sigma'(z_3) w_4 \sigma'(z_4) \frac{\partial C}{\partial a_4}$$



$$\frac{\partial C}{\partial b_1} = \sigma'(z_1) \underbrace{w_2 \sigma'(z_2)}_{< \frac{1}{4}} \underbrace{w_3 \sigma'(z_3)}_{< \frac{1}{4}} \underbrace{w_4 \sigma'(z_4) \frac{\partial C}{\partial a_4}}_{\text{common terms}}$$

$$\frac{\partial C}{\partial b_3} = \sigma'(z_3) \underbrace{w_4 \sigma'(z_4) \frac{\partial C}{\partial a_4}}_{\text{common terms}}$$

Deep neural networks

- Technologies for deep neural networks

Deep neural networks

- Technologies for deep neural networks
 - Convolution

Deep neural networks

- Technologies for deep neural networks
 - Convolution
 - Pooling

Deep neural networks

- Technologies for deep neural networks
 - Convolution
 - Pooling
 - GPU

Deep neural networks

- Technologies for deep neural networks
 - Convolution
 - Pooling
 - GPU
 - Reducing overfit
 - Expansion of training data
 - Dropout

Deep neural networks

- Technologies for deep neural networks
 - Convolution
 - Pooling
 - GPU
 - Reducing overfit
 - Expansion of training data
 - Dropout
 - Ensembles of networks

Deep neural networks

- Technologies for deep neural networks
 - Convolution
 - Pooling
 - GPU
 - Reducing overfit
 - Expansion of training data
 - Dropout
 - Ensembles of networks
- Recurrent neural networks